

**VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING**



**REPORT
CAPSTONE PROJECT
SEMESTER 252 ACADEMIC YEAR 2025-2026**

**DEVELOPING AN AI AGENT TO SUPPORT
ADMINISTRATION
AND OPERATION OF HPC SYSTEMS BASED ON
SLURM**

MAJOR: COMPUTER SCIENCE

SUPERVISOR(s): Assoc.Prof. Thoại Nam

—o0o—

STUDENT: Nguyễn Phúc Thanh Danh - 2252102

HO CHI MINH CITY, May 2026

Instructor's Signature

_____ Date: _____

Assoc.Prof. Thoai Nam (Instructor)

Faculty of Computer Science and Engineering

Declaration of Authenticity

We declare that we solely conducted this specialized project, under the supervision of Assoc.Prof. Thoai Nam at the Faculty of Computer Science and Engineering, Vietnam National University - Ho Chi Minh City University of Technology.

We have taken care to properly acknowledge and document all external sources and references used in the project.

If there is any instance of plagiarism, we are ready to accept the consequences. Ho Chi Minh City University of Technology - Vietnam National University HCMC will not be held responsible for any copyright violations that may have occurred during our research.

Ho Chi Minh City, May 2026

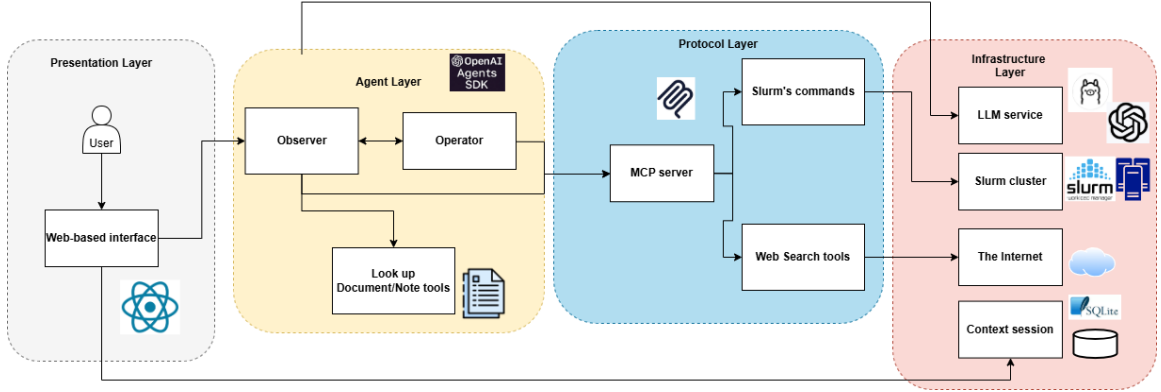
Authors,

Acknowledgement

We would like to express our sincere appreciation to Assoc.Prof. Thoại Nam for his invaluable guidance. Our research has greatly benefited from his deep knowledge, perceptive criticism, and constant support.

In addition, we would like to extend our gratitude to our families. Their unwavering faith in our abilities and constant encouragement have been our pillars of strength. Their belief in our potential has been a constant source of motivation and resilience. We are forever grateful for their love and support.

Abstract



Slurm Agent System Architecture Overview

High-Performance Computing (HPC) clusters have become essential infrastructure for scientific research, engineering simulations, and large-scale data processing. The Slurm Workload Manager is widely used across HPC environments and provides sophisticated resource management capabilities, but its command-line interface can create usability barriers for users who need to inspect queues, diagnose jobs, submit workloads, or perform controlled state-changing operations. This thesis addresses these challenges by designing and implementing an intelligent agent system that provides a natural language interface for Slurm cluster management.

The proposed Slurm Agent is a domain-specific control and assistance system for translating natural-language intent into concrete Slurm operations. Rather than treating an LLM as a general chatbot, the system combines typed Slurm tools, an Observer/Operator workflow with human confirmation for state-changing actions, local Slurm documentation retrieval, stateful confirmation handling, and trace-oriented evaluation. The implementation uses a React + Vite presentation layer, a session-aware FastAPI agent backend, an MCP protocol/tool layer, and real or mock Slurm infrastructure, but these frameworks serve as the substrate for Slurm-specific safety, grounding, and evaluation mechanisms.

This thesis makes six main contributions. **(1)** It defines a broad natural-language Slurm operation surface spanning monitoring, diagnosis, submission, job control, accounting, QoS/resource limits, reservations, scheduler health, configuration inspection, documentation support, and edge-case safety. **(2)** It implements a safety-centered Observer/Operator workflow that separates read-only analysis from state-changing execution through structured handoffs, human-in-the-loop confirmation, pending-action persistence, dynamic tool filtering, and session-scoped cleanup. **(3)** It develops a grounded Slurm assistance layer that combines live MCP command results for cluster state with local official-documentation retrieval for command syntax, state/reason-code interpretation, and policy-like explanations, reducing reliance on model memory alone. **(4)** It contributes a 3,135-case curated Slurm benchmark dataset with all eleven categories balanced at 285 cases each, including added coverage for fairshare, scheduler health,

configuration inspection, GPU/GRES behavior, job arrays, dependencies, QOS and resource-limit diagnosis, reservations, reports, licenses, topology, federation, burst buffers, triggers, local documentation retrieval, and administrative safety workflows. **(5)** It provides an architecture-aware evaluation stack and dashboard that run live agent requests against resettable mock scenarios, collect traces for tool usage, routing, confirmation behavior, response quality, and source-to-target state transitions, and optionally apply an LLM-as-judge component. **(6)** It explores domain-adapted fine-tuning of an open-weight model (Qwen2.5-14B) using LoRA adapters on a 4-bit quantized base (QLoRA) with role-scoped tool exposure, distilling the commercial model’s evaluation traces into a local alternative that eliminates API dependency while preserving tool-calling and safety behavior.

The implementation demonstrates the feasibility of a stateful, tool-grounded natural language interface for HPC operations. The fine-tuned open-weight model achieves 91.4% pass rate on the held-out benchmark, closing 78% of the gap to the commercial API baseline while eliminating external API dependency. Domain-adapted open-weight models serve as viable local alternatives for structured tool-calling tasks, reducing operational cost and preserving data sovereignty. The system provides a practical foundation for AI-assisted cluster operations where usability, traceability, and controlled execution must be balanced.

Keywords: High-Performance Computing, Slurm, Large Language Models, Model Context Protocol, Multi-Agent Systems, Human-in-the-Loop, Natural Language Interface, Parameter-Efficient Fine-Tuning

List of Abbreviations

Abbreviation	Description
API	Application Programming Interface
CLI	Command-Line Interface
CoT	Chain-of-Thought
CPU	Central Processing Unit
GPU	Graphics Processing Unit
HITL	Human-in-the-Loop
HPC	High-Performance Computing
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JSON-RPC	JavaScript Object Notation Remote Procedure Call
LLM	Large Language Model
LoRA	Low-Rank Adaptation
MCP	Model Context Protocol
NLP	Natural Language Processing
PEFT	Parameter-Efficient Fine-Tuning
QLoRA	Quantized Low-Rank Adaptation
RAG	Retrieval-Augmented Generation
ReAct	Reasoning and Acting
REST	Representational State Transfer
RLHF	Reinforcement Learning from Human Feedback
SDK	Software Development Kit
SQL	Structured Query Language
Slurm	Simple Linux Utility for Resource Management
SSE	Server-Sent Events
TLS	Transport Layer Security
UI	User Interface
URI	Uniform Resource Identifier

Contents

Abstract	iv
List of Abbreviations	vi
1 Introduction	1
1.1 Motivation	1
1.2 Scope and Goals	2
1.2.1 Project Goals	2
1.3 Challenges and Solutions	3
1.4 Project Structure	5
2 Theoretical Background	7
2.1 Slurm Workload Manager	7
2.1.1 Architecture Overview	7
2.1.2 Core Operational Objects	8
2.1.3 Why Slurm Is Difficult to Use Through Natural Language	9
2.2 Large Language Models	9
2.2.1 Transformer-Based Language Modeling	10
2.2.2 Instruction Following and Alignment	11
2.2.3 Tool Use and Structured Output	11
2.2.4 Retrieval-Augmented Generation	12
2.2.5 Limitations in Operational Settings	13
2.2.6 Parameter-Efficient Fine-Tuning	13
2.3 Model Context Protocol	14
2.3.1 Protocol Overview	14
2.3.2 Core Components and Capabilities	16
2.3.3 Tool Definition and Deployment	16
2.3.4 Security Considerations	16
2.4 Agent Architectures and Frameworks	17
2.4.1 ReAct: Reasoning and Acting	17
2.4.2 OpenAI Agents SDK	18
3 Related Tools and Works	20
3.1 HPC Management Interfaces	20
3.2 LLM-Based HPC Systems	20
3.3 LLM Agents and Tool-Augmented Reasoning	20
3.4 Protocol-Based Tool Integration	21
3.5 Natural Language to Structured Operations	21

3.6	Safety and Evaluation	21
4	Proposed Approaches	22
4.1	Proposed Slurm Agent Overview	22
4.1.1	System Objective	22
4.2	Requirements Overview	23
4.2.1	Functional Requirements	23
4.2.2	Non-Functional Requirements	24
4.2.3	Requirements Traceability	24
4.2.4	Constraints	25
4.3	System Architecture	25
4.3.1	Architecture Overview	25
4.3.2	Design Rationale	26
4.3.3	Component Descriptions	27
4.3.4	Data Flow and Deployment	30
4.4	Module Specifications	30
4.4.1	Observer Module	30
4.4.2	Operator Module	31
4.4.3	MCP Tool Layer	31
4.4.4	Documentation Retrieval	32
4.4.5	Evaluation Pipeline	32
4.5	Large Language Model Architecture	32
4.5.1	ReAct Reasoning Loop	32
4.5.2	Instruction Design	33
4.5.3	Domain-Adapted Fine-Tuning	33
5	Implementation	38
5.1	Agent Backend Implementation	38
5.1.1	Technology Stack	38
5.1.2	Agent Topology	38
5.1.3	Safety and Confirmation Pipeline	39
5.1.4	Hybrid RAG Documentation Retrieval	39
5.1.5	Runtime Services	40
5.1.6	Fine-Tuned Model Serving	40
5.2	MCP Server Implementation	41
5.2.1	Design Principles	41
5.2.2	Tool Surface	41
5.2.3	Dual Execution Modes	42
5.2.4	Web Search	42
5.2.5	Transport	42
5.3	Runtime Interface and Evaluation Integration	43

5.3.1	Interface Role in the System	43
5.3.2	Additional Capability Screenshots	45
5.3.3	Agent Service API	47
5.3.4	Evaluation Integration	47
5.4	Fine-Tuning Implementation	48
5.4.1	Model Preparation	48
5.4.2	Data Formatting	48
5.4.3	Training Procedure	49
5.4.4	Training Monitoring	49
6	Experiments	51
6.1	Testing Approach	51
6.1.1	Testing Environment	51
6.1.2	Benchmark Dataset	51
6.1.3	Train/Test Split for Fine-Tuning Evaluation	53
6.1.4	Scoring Methodology	54
6.1.5	Execution Flow	54
6.1.6	Source Code	54
6.2	Experimental Results	54
6.2.1	Benchmark Construction Outcome	54
6.2.2	Dataset Construction Outcome	55
6.2.3	Training Data Construction and Validation	55
6.2.4	Comparison with Manual CLI Workflow	57
6.2.5	Quantitative Performance Results	57
6.2.6	Model Configuration	58
6.2.7	Fine-Tuned Model vs Baseline Comparison	58
6.2.8	Result Interpretation	61
6.2.9	Architecture Ablation: Monolithic vs. Observer/Operator	62
7	Conclusion	66
7.1	Summary	66
7.1.1	Research Contributions	66
7.1.2	Achievement Against Project Goals	67
7.1.3	Main Findings	67
7.1.4	Source Code Availability	68
7.2	Limitations	68
7.2.1	Evaluation Scope	68
7.2.2	Evaluation–Architecture Coupling	68
7.2.3	Behavioral and Deployment Constraints	68
7.2.4	Model and Metric Constraints	69

List of Tables

3.1	Comparison of LLM-Based HPC Systems	20
4.1	Functional Requirements	23
4.2	Non-Functional Requirements	24
4.3	Requirements Traceability to Project Goals	25
4.4	MCP Tool Groups	31
5.1	MCP Server Tool Reference	42
5.2	Fine-Tuning Hyperparameters	49
6.1	Test Environment Specifications	51
6.2	Benchmark Categories	52
6.3	Constructed Benchmark Summary	55
6.4	Training Data Summary	55
6.5	Training Run Configuration	56
6.6	Task Complexity: Agent vs Manual CLI	57
6.7	Aggregate Evaluation Metrics — GPT-5-mini Baseline (3,135 Cases, Rescored)	57
6.8	Evaluation Model Stack	58
6.9	Three-Way Model Comparison (615 Test Cases)	59
6.10	Pass Rate by Category — Three-Way Comparison (615 Test Cases)	60
6.11	Deployment Cost Comparison	60
6.12	Architecture Ablation — Base Qwen2.5-14B (615 Test Cases)	62
6.13	Per-Category Pass Rate — Monolithic Ablation	62
6.14	Routing-Neutral Comparison (378 Observer-Only Cases)	64

List of Figures

2.1	Slurm master-worker architecture	8
2.2	Transformer encoder-decoder architecture [42]	10
2.3	Full fine-tuning vs. LoRA vs. QLoRA [10]	14
2.4	Model Context Protocol (MCP) architecture [16]	15
2.5	The ReAct reasoning loop [48]	18
4.1	Slurm Agent system architecture (four-layer stack)	26
4.2	Tool partitioning: Observer/Operator split vs. Monolithic	28
4.3	Observer/Operator routing and HITL confirmation flow	29
4.4	Request lifecycle data flow	30
4.5	ReAct reasoning cycle (Think–Act–Observe)	33
4.6	MCP mock environment for training data collection	34
4.7	Training data augmentation pipeline	36
5.1	HITL Flow (1/3) — Read-only query	43
5.2	HITL Flow (2/3) — Confirmation intercept	44
5.3	HITL Flow (3/3) — Post-execution verification	44
5.4	Web Search (1/2) — Tool execution phase	45
5.5	Web Search (2/2) — Final response	46
5.6	Documentation Lookup (1/2) — Retrieval phase	46
5.7	Documentation Lookup (2/2) — Final response	47
6.1	Dataset Distribution by Category (3,135 total cases)	52
6.2	Dataset Distribution by Scenario	53
6.3	Category $\times$ Scenario Heatmap	53
6.4	Per-Category Pass Rate: Observer/Operator vs. Monolithic Architecture (615 Test Cases)	63
6.5	LLM judge score: three configurations (615 test cases)	63
6.6	Routing-neutral comparison: Observer-only tasks	64

Chapter 1

Introduction

1.1 Motivation

High-Performance Computing (HPC) has become an essential infrastructure in modern scientific research, engineering simulations, and large-scale data processing. HPC clusters enable researchers to tackle computationally intensive problems that would be infeasible on conventional computing systems. From climate modeling and genomics to machine learning and financial simulations, HPC systems provide the computational power necessary to drive innovation across multiple domains.

Slurm Workload Manager, originally introduced as the Simple Linux Utility for Resource Management, has emerged as a widely deployed workload manager for HPC clusters [49, 37]. SchedMD reports that Slurm is used at government laboratories, universities, and companies worldwide, and that it manages workloads for over half of the top 10 systems in the TOP500 list [35]. Slurm provides sophisticated mechanisms for job scheduling, resource allocation, and cluster monitoring, enabling efficient utilization of computational resources across thousands of nodes. The system’s flexibility and scalability have made it the preferred choice for many academic institutions and commercial organizations operating large-scale computing facilities.

Despite its power and flexibility, HPC user-support literature highlights recurring needs for training, documentation, and assistance when researchers interact with complex computing environments [34]. In Slurm-based clusters, this challenge is reflected in a broad command-line interface (CLI) comprising numerous commands, each with extensive sets of options and parameters [37]. Users must master commands such as `squeue`, `sinfo`, `sbatch`, `scontrol`, and `sacct`, along with their associated flags and output formats. This complexity is compounded by the need to write job submission scripts in Bash, which requires additional technical knowledge beyond the domain expertise of many researchers.

The learning curve associated with Slurm administration and usage creates several operational challenges. System administrators spend considerable time training new users, while researchers face delays in their work as they navigate the intricacies of job submission and monitoring. Common tasks such as checking job status, understanding resource availability, or debugging failed jobs often require consulting extensive documentation or seeking expert assistance.

Recent advances in Large Language Models (LLMs) have demonstrated remarkable capabilities in natural language understanding and generation [7, 25]. These models can interpret complex user queries, reason about multi-step tasks, and generate appropriate responses or actions. The emergence of LLM-based agents—systems that combine language models with tool-calling capabilities—has opened new possibilities for creating intelligent interfaces to complex

systems [47, 38].

The Model Context Protocol (MCP), developed by Anthropic, provides a standardized approach for connecting LLMs to external tools and data sources [4, 5]. MCP enables the creation of modular, reusable integrations that can expose system functionality to AI agents through explicit tool interfaces. This architectural pattern separates the concerns of tool implementation from agent logic, facilitating the development of sophisticated AI-assisted applications.

However, deploying commercial LLM APIs in HPC environments raises fundamental concerns. Cluster workloads expose sensitive metadata—user submission scripts, resource allocation patterns, job failure diagnostics, and access-control policies—that institutional data governance policies prohibit from traversing external networks. Furthermore, recurring per-query API costs scale linearly with cluster user-base size. These constraints motivate the deployment of locally-hosted open-weight models that preserve data sovereignty and eliminate ongoing costs, but at the expense of model capacity: a 14-billion parameter model cannot match the tool-selection accuracy of commercial frontier models when presented with a large number of available tools simultaneously.

This research is motivated by the opportunity to bridge the gap between Slurm’s powerful capabilities and user accessibility through an intelligent conversational interface that operates entirely on local infrastructure. By leveraging LLM agents connected to Slurm through MCP, and by partitioning the tool surface across specialized sub-agents to compensate for the capacity constraints of local models, we create a system that allows users to manage cluster resources using natural language queries while maintaining full data sovereignty.

1.2 Scope and Goals

1.2.1 Project Goals

The primary objective of this research is to design, implement, and evaluate an intelligent agent system for Slurm cluster management. The project aims to make Slurm more accessible through natural-language interaction, but it also contributes the evaluation infrastructure needed to measure whether such an agent behaves correctly and safely. The following goals guide the development of this solution:

Goal 1: Natural Language Interface for Broad Slurm Operations. The system shall enable users to perform a broad set of Slurm operations through natural language queries. Users should be able to inspect jobs, partitions, nodes, accounting records, QoS and resource limits, reservations, scheduler health, submit workloads, and perform human-confirmed job-control operations without explicit knowledge of Slurm command syntax. The agent must interpret user intent accurately and translate queries into appropriate Slurm commands or documentation lookups.

Goal 2: Slurm-Specific Observer/Operator Control Flow. The system shall implement a specialized agent workflow where read-only observation is separated from state-changing job

management. Documentation retrieval, data visualization, and web search fallback should remain read-only support capabilities, while handoffs to state-changing execution must carry explicit action intent, target scope, and safety context.

Goal 3: Risk-Aware Slurm Tool Integration via MCP. The system shall utilize the Model Context Protocol to provide standardized tool interfaces between the LLM agents and Slurm infrastructure. The MCP server should expose Slurm functionality as discrete, typed tools, while the agent layer classifies tools by operational risk and routes state-changing actions through confirmation before command execution.

Goal 4: Curated Slurm Benchmark Dataset. The project shall construct a broad, scorable Slurm benchmark dataset for evaluating natural-language cluster operations. The active dataset shall cover monitoring, diagnosis, action, bulk operations, submission, safety, accounting, documentation, domain-specific Slurm workflows, multi-step reasoning, and edge cases. Each case should include expected tool behavior, routing expectations, confirmation policy, and source-to-target state information where applicable. The current active benchmark contains 3,135 curated cases, balanced across eleven categories with 285 cases each.

Goal 5: Domain-Adapted Fine-Tuning. The project shall explore whether a smaller open-weight model can be domain-adapted to replicate the tool-calling and safety behavior of the commercial baseline. The approach uses parameter-efficient fine-tuning—LoRA adapters trained on a 4-bit quantized base model (QLoRA)—on traces distilled from the evaluation framework, with role-scoped tool exposure to prevent cross-role hallucination. The resulting model should operate entirely on local infrastructure, eliminating API cost and data-privacy concerns for production HPC environments.

1.3 Challenges and Solutions

Reliability and Accuracy

Challenge: Ensuring reliability and accuracy in a natural-language Slurm agent is difficult because Slurm commands depend on exact syntax, object identifiers, flags, partitions, accounts, and current scheduler state. A generated answer may sound plausible while still selecting the wrong tool, referring to the wrong job, misunderstanding a pending reason, or confusing documentation knowledge with live cluster facts.

Solutions: The system grounds user requests in typed MCP tools instead of free-form shell generation. The agent uses a ReAct-style execution pattern [48], but the important project-specific layer is the Slurm grounding around it: live state is retrieved through Slurm tools, command and state explanations are supported by local official-documentation retrieval, and under-specified requests can trigger clarification rather than forced execution.

Safety in Cluster Operations

Challenge: Slurm is a shared operational environment. Read-only inspection is low risk, but cancelling jobs, holding jobs, changing node state, modifying reservations, or updating accounting state can interrupt users and affect shared resources. These actions cannot be handled

like ordinary chatbot responses.

Solutions: The system separates read-only observation from state-changing operation through an Observer/Operator workflow. The Observer handles inspection, diagnosis, accounting/resource-limit queries, reservation checks, documentation retrieval, and web-search fallback. The Operator handles state-changing work through structured handoffs and a pending-action workflow that requires explicit user confirmation before execution. Dynamic tool filtering keeps sub-agents within their intended responsibility boundaries, and MCP parameter validation prevents arbitrary shell-command construction.

Data Construction for Evaluation

Challenge: Evaluating an agentic Slurm assistant requires more than a small list of example prompts, but there is no ready-made public dataset that covers the exact Slurm workflows and tool behavior needed for this project. The benchmark data therefore has to be manually designed as synthetic scenarios. It must cover read-only monitoring, diagnosis, submission, job control, accounting, documentation questions, multi-step workflows, and edge cases, while remaining scorable through clear expectations for tools, routing, confirmation, response evidence, and state changes where applicable.

Solutions: The project manually constructs a synthetic 3,135-case scenario-grid benchmark across eleven balanced categories. Each case specifies expected tool behavior, routing behavior, confirmation policy, keywords, and source-to-target state behavior where applicable. The dataset is validated against supported tools and mock Slurm state so that benchmark failures can be interpreted as agent behavior rather than inconsistent ground truth.

Diversity of Slurm Workflows

Challenge: Slurm administration spans many domains, including jobs, nodes, partitions, accounting, QoS, fair-share, reservations, configuration, scheduler health, job arrays, dependencies, licenses, reports, topology, and selected administrative actions. A narrow prototype using only `squeue` and `scancel` would not represent the operational breadth expected from an HPC assistant.

Solutions: The MCP tool layer exposes a broad Slurm operation surface, and the benchmark mirrors that breadth across balanced categories. Live or mock MCP command results remain the source of truth for cluster-state questions, while local Slurm documentation retrieval supports reference-heavy questions. This keeps live operational facts separate from documentation knowledge and reduces dependence on model memory alone.

State and Context Isolation

Challenge: Conversation state is useful for real users, but it can contaminate automated evaluation if previous test cases leak into later cases. Follow-up prompts need session memory, yet benchmark cases must start from clean context and known source state. Large artifacts or verbose traces can also pollute future model context if stored as ordinary conversation text.

Solutions: The backend maintains session-scoped conversation state and pending actions for interactive use, while evaluation runs use deterministic isolated sessions. Mock Slurm scenarios can be reset before each case, and parallel evaluation workers use separate session iden-

tifiers and worker-specific MCP endpoints. Chart artifacts and large generated payloads are separated from ordinary model context to reduce unnecessary token usage.

Traceable Agent Evaluation

Challenge: Text-only evaluation is insufficient for a tool-using agent. A response can explain the correct operation while still using the wrong tool path, skipping a required handoff, failing to request confirmation, or producing the wrong state transition in the simulator.

Solutions: The evaluator sends prompts through the live backend, records tool and routing traces, applies confirmation logic, and checks structural metrics such as tool recall, routing match, HITL behavior, keyword coverage, and state matching. This makes failures inspectable and repeatable, allowing the project to measure architecture-level behavior rather than relying only on final response fluency.

Domain-Adapted Fine-Tuning

Challenge: Relying on commercial API models introduces per-request cost, latency, and data-privacy concerns for production HPC environments. However, smaller open-weight models suffer from two compounding limitations: (1) they do not natively understand Slurm tool-calling conventions, and (2) their tool-selection accuracy degrades when the number of available tools exceeds approximately 10—a phenomenon documented in recent tool-use scaling studies [31]. The Slurm agent exposes 18 distinct tools; presenting all of them to a single 14B-parameter model leads to selection confusion, where the model calls `sacct` when `squeue` is appropriate or hallucinates non-existent commands such as `scontrol_requeue`.

Solutions: The project addresses both limitations through a dual strategy. First, the Observer/Operator architecture partitions the 18-tool surface into two focused subsets—approximately 10 read-only tools for the Observer and approximately 8 action tools for the Operator—reducing each agent’s selection space and recovering tool-recall accuracy (empirically +11.6 percentage points on routing-neutral tasks, Section 6.2.9). Second, the project distills successful evaluation traces from the commercial model into a structured training corpus, then applies parameter-efficient fine-tuning—LoRA adapters on a 4-bit quantized base (QLoRA)—to an open-weight model (Qwen2.5-14B). Role-scoped tool exposure ensures each training sample includes only the tools available to its role, so the model internalizes both the safety boundary and the reduced tool surface as part of its generation behavior. The resulting model runs entirely on local GPU infrastructure with no external API dependency.

1.4 Project Structure

The report is divided into seven chapters, with a brief overview of each chapter as follows:

- **Chapter 1 - Introduction:** This chapter introduces the project topic, explains the motivation for a natural-language Slurm assistant, defines the scope and objectives, discusses the main challenges, and outlines the adopted solutions.
- **Chapter 2 - Theoretical Background:** This chapter presents the foundational knowledge

required for the project, including Slurm workload management, Large Language Models, agent architectures, and the Model Context Protocol.

- **Chapter 3 - Related Tools and Works:** This chapter reviews existing HPC management interfaces, LLM agent frameworks, and MCP-based tool integrations, then positions the project relative to those works.
- **Chapter 4 - Proposed Approaches:** This chapter analyzes and designs the Slurm Agent system, including requirements, system architecture, component responsibilities, and the Observer/Operator workflow.
- **Chapter 5 - Implementation:** This chapter describes the implementation of the main system components: the MCP Slurm tool server, the agent backend, the frontend interface, and supporting evaluation infrastructure.
- **Chapter 6 - Experiments:** This chapter presents the testing approach, benchmark design, evaluation criteria, and experimental results of the Slurm Agent system.
- **Chapter 7 - Conclusion:** This chapter summarizes the project outcomes, discusses limitations, and reflects on the contributions relative to the project goals.

Chapter 2

Theoretical Background

2.1 Slurm Workload Manager

Slurm Workload Manager is an open-source cluster management and job scheduling system for Linux-based high-performance computing environments [49, 37]. It is widely deployed in research and production HPC clusters, including large systems represented in the TOP500 list [35]. This project uses Slurm as the target operational domain because it exposes a rich command surface for monitoring, diagnosis, submission, accounting, and controlled cluster administration.

2.1.1 Architecture Overview

Slurm follows a controller-and-worker architecture. The central controller maintains cluster state and scheduling decisions, while node-level daemons execute and monitor workloads.

The **slurmctld** daemon is the central controller. It tracks nodes, partitions, jobs, reservations, and resource allocations. It accepts job submissions, evaluates pending work, applies scheduling policy, and dispatches jobs to available compute resources. In production clusters, a backup controller can be configured for high availability.

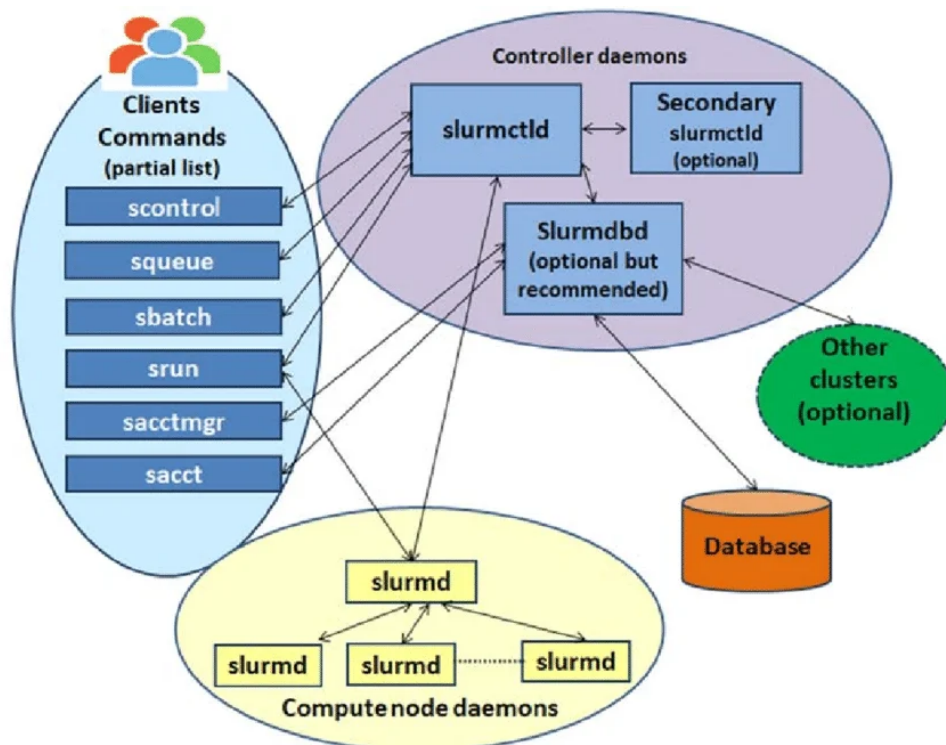


Figure 2.1: Slurm master-worker architecture. Client commands (`sbatch`, `srun`, `squeue`) submit requests to the central `slurmctld` controller daemon, which schedules work and dispatches it to per-node `slurmd` daemons on compute nodes. An optional `slurmdbd` accounting daemon records job history in a persistent database for later retrieval via `sacct`. A secondary `slurmctld` can be configured for high availability, and the controller may communicate with external clusters for multi-cluster management [1].

The **`slurmd`** daemon runs on each compute node. It launches job steps, monitors local execution, reports node state to the controller, and handles job control signals. This makes the compute node visible to the scheduler while preserving local execution responsibility.

The **`slurmdbd`** daemon provides the accounting service when enabled. It stores job history, user associations, account usage, fair-share information, and reporting data in a database. These records are important for administrative workflows, quota analysis, and historical job diagnosis.

2.1.2 Core Operational Objects

Slurm operations are built around several core objects:

Nodes are compute servers with resources such as CPUs, memory, GPUs, and generic resources. Node state matters operationally: an idle node can accept work, an allocated node is running jobs, a drained node is intentionally unavailable for new work, and a down node is unavailable because of failure or administrative action.

Partitions are logical groups of nodes. They behave like queues and encode access rules, limits, default policies, and intended usage patterns. Users often need to know which partition matches a job’s resource requirements and policy constraints.

Jobs are resource allocation requests. A job includes requested resources, time limits, partition, account or QoS settings, script or command, and optional dependencies. Jobs move through states such as pending, running, completed, failed, cancelled, held, or suspended.

Steps are execution units inside an allocated job. A job may contain one or more steps, and step-level inspection is useful for understanding running work, CPU usage, memory behavior, and launched parallel tasks.

Accounts, QoS, and reservations represent policy and access constraints. They affect who can submit work, how resources are charged, which jobs receive priority, and when nodes are reserved for maintenance or special workloads.

These objects are accessed through a broad command surface. Read-only commands such as `sinfo`, `squeue`, `sacct`, and `scontrol show` inspect partitions, jobs, accounting records, nodes, reservations, and configuration state. Other commands, including `sbatch`, `srun`, `salloc`, `scancel`, selected `scontrol` operations, and `sacctmgr`, can submit work or modify jobs, nodes, reservations, accounts, and QoS policies. The same command family may contain both safe inspection operations and high-impact administrative operations; for example, `scontrol show job` is read-only, while `scontrol update` can change job or node state. This distinction is central to the Observer/Operator design used later in the thesis.

2.1.3 Why Slurm Is Difficult to Use Through Natural Language

Slurm’s command-line interface is expressive, but that expressiveness creates usability and safety challenges. Users must know command names, flags, output formats, state meanings, pending reasons, account/QoS concepts, and site-specific policy conventions. Many operations are also state-dependent: cancelling a running job, releasing a held job, or updating a reservation only makes sense when the referenced object exists and is in an appropriate state.

These properties make Slurm a useful but demanding domain for a natural-language agent. The agent must map user intent to the correct command surface, obtain live cluster facts when needed, distinguish documentation questions from operational questions, and require explicit confirmation before state-changing actions are executed.

2.2 Large Language Models

Large Language Models (LLMs) are neural networks trained to model and generate natural language from large text corpora. Recent LLMs can follow instructions, summarize technical material, reason over multi-step requests, and call external tools, which makes them useful as natural-language interfaces for complex systems [7, 25]. In this thesis, the LLM is not treated as an authority on live cluster state. Instead, it is used as an intent interpretation and orchestration component that must be grounded through external tools, documentation, and structured responses.

2.2.1 Transformer-Based Language Modeling

Most modern LLMs are based on the Transformer architecture introduced by Vaswani et al. [42]. The key mechanism is self-attention, which allows each token representation to attend to other tokens in the sequence and model long-range dependencies.

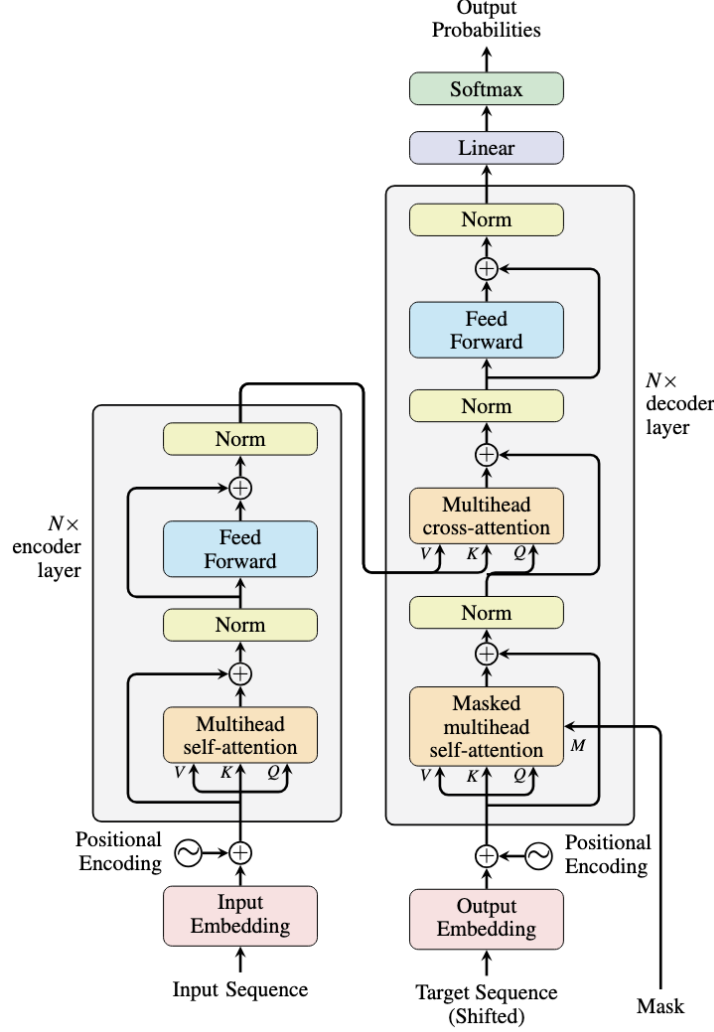


Figure 2.2: Transformer architecture as introduced by Vaswani et al. [42]. The encoder (left) and decoder (right) each consist of stacked layers combining multi-head self-attention with position-wise feed-forward networks, residual connections, and layer normalisation. Inputs are first converted to token embeddings and summed with positional encodings to inject sequence-order information. The encoder builds contextual representations of the input; the decoder attends to both its own previous outputs (masked self-attention) and the encoder output (cross-attention) to generate the target sequence token by token. Although the original design is encoder-decoder, most modern LLMs use a decoder-only variant of this structure.

In self-attention, token representations are projected into query (Q), key (K), and value (V) vectors. The attention output is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2.1)$$

where d_k is the key-vector dimension. Multi-head attention performs this computation across several learned projections, allowing different heads to capture different relationships in the text. Stacked attention and feed-forward layers then produce contextual representations used for next-token prediction.

Most LLMs are trained with an autoregressive objective, where the model predicts the next token from the preceding context:

$$P(x) = \prod_{i=1}^n P(x_i | x_1, \dots, x_{i-1}) \quad (2.2)$$

This objective allows the model to learn linguistic patterns, factual associations, programming conventions, and domain terminology from training data. However, the generated output remains probabilistic. A fluent answer can still be incomplete, stale, or unsupported by the current state of an external system.

2.2.2 Instruction Following and Alignment

Base language models are optimized for continuation, not necessarily for helpful task completion. Instruction tuning improves this behavior by training models on instruction-response examples. Reinforcement Learning from Human Feedback (RLHF) further adjusts model behavior according to human preferences for helpfulness, relevance, and safety [27]. Other alignment approaches, such as Constitutional AI, use explicit principles to critique and revise model outputs [6].

These techniques make LLMs better suited for conversational systems, but they do not remove the need for domain constraints. In a Slurm environment, an apparently correct response may still use the wrong partition, assume outdated job state, or omit an operational precondition. Therefore, alignment must be combined with tool grounding, parameter validation, and confirmation for actions that affect cluster state.

2.2.3 Tool Use and Structured Output

Tool use extends an LLM beyond static model knowledge. Instead of answering only from its parameters, the model can choose an external function, provide structured arguments, inspect the returned result, and continue the conversation using that result. This pattern is important for operational domains because live state must be queried from the system of record.

Function calling and structured output make tool use more reliable than free-form command generation. A developer defines available tools with names, descriptions, argument schemas, and required fields. The model then emits a structured call that software can validate before execution. For example, a natural-language request such as checking a user’s queued jobs can be mapped to a job-query tool rather than a raw shell string.

Structured tool calls support three important properties in this thesis:

- **Grounding:** the agent can retrieve live Slurm state instead of relying on memorized or guessed information.
- **Control:** the application can restrict the available tool set according to the current interaction mode.
- **Validation:** typed parameters can be checked before a command is executed against the cluster interface.

2.2.4 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) addresses a core limitation of LLMs: their parametric knowledge is frozen at training time and cannot reflect new documentation, version-specific syntax, or domain-specific reference material [20]. RAG augments the generative model with a retrieval step that fetches relevant passages from an external corpus before generation, grounding the output in authoritative source material.

A typical RAG pipeline consists of three stages:

1. **Indexing:** documents are split into chunks and encoded into dense vector representations using a pre-trained embedding model such as Sentence-BERT [33].
2. **Retrieval:** at query time, the user query is embedded with the same model and the top- k most similar chunks are retrieved via cosine similarity.
3. **Generation:** the retrieved chunks are prepended to the LLM prompt as context, and the model generates an answer grounded in those passages.

Lexical vs. Semantic Retrieval

Lexical retrieval (e.g., BM25, TF-IDF) matches based on token overlap between query and document. It is fast and effective for keyword-rich queries but fails when the user paraphrases concepts without using the exact terminology present in documents.

Semantic retrieval encodes both queries and documents into a shared embedding space where cosine similarity captures meaning rather than surface form. For example, the query “how to chain jobs so one runs after another” can match documentation about `--dependency=afterok` even though no keywords overlap.

Hybrid Retrieval and Reciprocal Rank Fusion

Neither method dominates across all queries. Hybrid retrieval combines both by fusing their ranked result lists. Reciprocal Rank Fusion (RRF) [9] merges rankings without requiring score normalization:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)} \quad (2.3)$$

where R is the set of retrieval systems and k is a smoothing constant (typically 60). Documents that rank highly in both lexical and semantic systems receive the highest fused score, while documents that appear in only one system are still surfaced.

This approach is well-suited for operational documentation retrieval where queries range from exact command names (“sbatch –parsable”) to conceptual questions (“why is my job stuck”).

2.2.5 Limitations in Operational Settings

LLMs introduce risks that are especially relevant to HPC operations. They may hallucinate non-existent commands, merge similar options from different tools, provide outdated documentation, or infer an action that the user did not explicitly approve. They may also produce plausible explanations for failures without actually checking logs, accounting records, or scheduler state.

For these reasons, the agent architecture in this project separates language understanding from operational execution. The LLM interprets intent and decides which typed tool may be relevant, while external tools provide authoritative data and the application enforces confirmation before state-changing operations. This design uses the LLM for its strengths in language and orchestration while limiting its authority over the cluster.

2.2.6 Parameter-Efficient Fine-Tuning

Full fine-tuning of large language models updates all parameters, requiring GPU memory proportional to the full model size. For models with billions of parameters, this is impractical on commodity hardware. Parameter-Efficient Fine-Tuning (PEFT) methods address this by modifying only a small subset of parameters while keeping the base model frozen.

Low-Rank Adaptation (LoRA) [17] decomposes weight updates into low-rank matrices. For a pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times k}$, LoRA adds a trainable decomposition:

$$W = W_0 + \Delta W = W_0 + BA \quad (2.4)$$

where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ with $\text{rank } r \ll \min(d, k)$. Only A and B are trained, reducing trainable parameters from $d \times k$ to $r \times (d + k)$. A scaling factor α/r controls the magnitude of the adaptation. At inference time, the low-rank matrices can be merged back into the base weights with zero additional latency.

Quantized LoRA (QLoRA) [10] extends LoRA by quantizing the frozen base model to 4-bit precision (NormalFloat4) during training. The LoRA adapters remain in higher precision (bfloat16) for gradient computation. This reduces the memory footprint of the base model by approximately $4\times$, enabling fine-tuning of 14B-parameter models on a single 48 GB GPU.

QLoRA introduces double quantization—quantizing the quantization constants themselves—for additional memory savings with negligible quality loss.

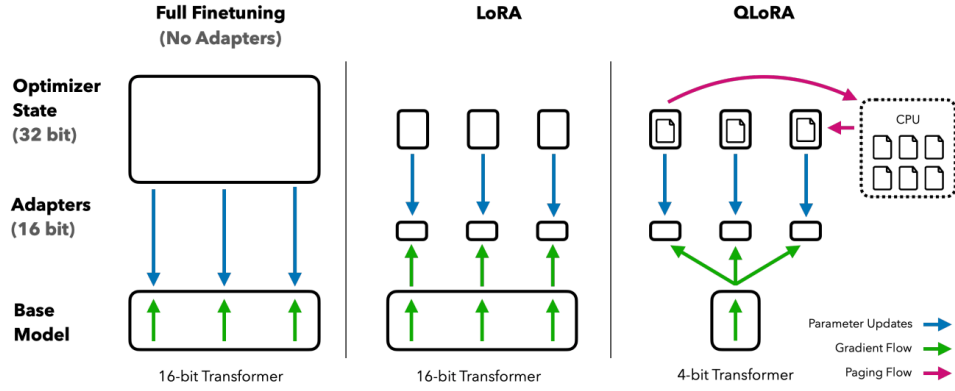


Figure 2.3: Comparison of fine-tuning approaches: (left) full fine-tuning updates all 16-bit weights with 32-bit optimizer state; (center) LoRA freezes the base model and trains low-rank adapters; (right) QLoRA quantizes the frozen model to 4-bit and uses paged optimizers to manage memory spikes [10]

Knowledge Distillation transfers capabilities from a larger teacher model to a smaller student model. In the context of tool-calling agents, distillation can use the teacher’s successful execution traces as training data: the student learns to reproduce the teacher’s tool selections, argument formatting, and response patterns without requiring manually annotated data [15]. This is particularly effective when the teacher model’s behavior is already validated through an evaluation framework.

2.3 Model Context Protocol

The Model Context Protocol (MCP) is an open protocol for connecting AI applications to external tools, resources, and reusable prompts [4, 5]. It provides a standard boundary between a language model application and the systems it needs to inspect or operate. In this thesis, MCP is important because it allows Slurm operations to be exposed as typed tools instead of free-form shell commands.

2.3.1 Protocol Overview

MCP addresses a common integration problem in AI applications. Without a shared protocol, each application must implement its own custom connection to every database, API, command-line tool, or document source. This makes integrations difficult to reuse and makes behavior harder to inspect.

MCP uses a client-server model. A server declares the capabilities it provides, and a client connects those capabilities to an AI host application. Communication is based on JSON-RPC 2.0, which gives the protocol a structured request-response format that can be implemented across different programming languages.

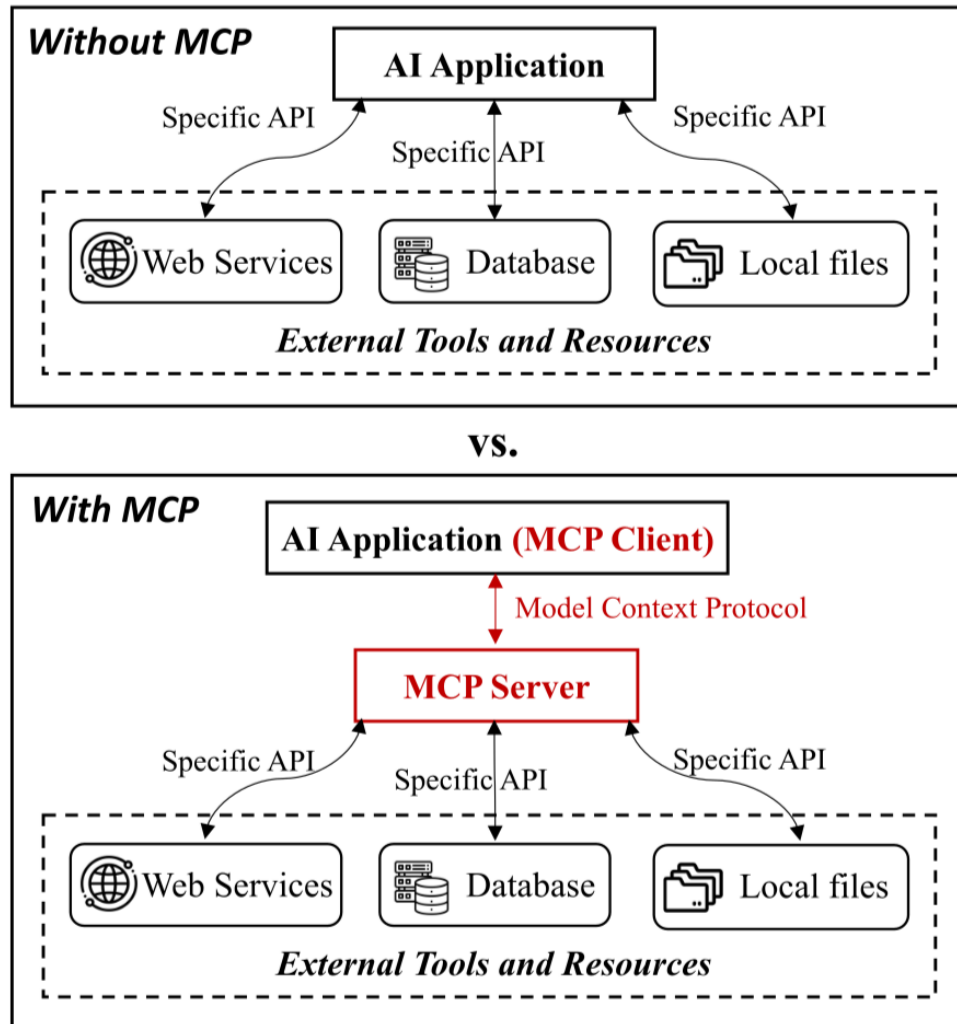


Figure 2.4: Model Context Protocol (MCP) architecture, introduced by Anthropic in late 2024 [16]. *Without MCP* (top), every AI application must implement a separate, bespoke integration for each data source or tool—a fragmented approach that is costly to build and brittle to maintain. *With MCP* (bottom), a standardised client-server protocol acts as a universal gateway: the AI application runs an **MCP Client** that communicates with one or more **MCP Servers**, each of which exposes a specific data source or capability (databases, file systems, web services, command-line tools) through a common interface. Once a server is built it is immediately compatible with any MCP-capable host, eliminating per-integration custom code and enabling a reusable, composable tool ecosystem.

The protocol does not define the internal logic of a tool. Instead, it defines how the tool is described, discovered, invoked, and returned to the host application. This makes MCP suitable for wrapping existing systems such as Slurm command-line utilities.

2.3.2 Core Components and Capabilities

MCP architecture contains three main roles:

- **MCP Hosts:** applications that users interact with directly, such as chat interfaces, IDE extensions, or desktop assistants.
- **MCP Clients:** protocol clients that maintain connections between the host and one or more servers, performing capability discovery and forwarding requests.
- **MCP Servers:** domain-specific services that expose capabilities. In this project, a Slurm MCP server exposes selected scheduler operations as named tools with defined input schemas.

Servers can expose three capability categories: **Resources** (readable data such as logs or status), **Tools** (executable operations with typed schemas), and **Prompts** (reusable workflow templates). For the Slurm Agent, tools are the primary capability—they allow Slurm operations to be represented as explicit function calls rather than free-form shell commands.

2.3.3 Tool Definition and Deployment

MCP tools are described with a name, a natural-language description, and a JSON Schema input definition. When the model selects a tool, the client sends a `tools/call` request; the server validates inputs, executes the underlying operation, and returns a structured result or error. This avoids asking the model to construct arbitrary shell commands: instead of generating a raw `squeue` string, the model calls a typed job-listing tool with validated filters. Read-only tools can be made available for observation while state-changing tools are separated into a different interaction mode requiring user confirmation. MCP supports both local deployment (server launched as a subprocess over `stdio`) and network deployment (HTTP transport), without changing the logical tool interface.

2.3.4 Security Considerations

MCP provides structure, but it does not automatically make tool use safe. Security must be enforced by the server and the host application. Important concerns include:

- **Access control:** only expose operations that the current user or deployment is allowed to perform.
- **Input validation:** reject missing, malformed, or out-of-policy arguments before command execution.
- **Output control:** avoid exposing sensitive user, job, or accounting information unnecessarily.

- **Auditability:** log tool invocations and results for debugging and accountability.
- **User confirmation:** require explicit approval before actions that modify cluster state.

These concerns are especially important for HPC systems because a single operation can affect shared resources, queued jobs, and other users. Therefore, this project uses MCP as the structured integration layer, while confirmation flow and mode separation provide additional control over state-changing Slurm operations.

2.4 Agent Architectures and Frameworks

Building intelligent agents requires combining Large Language Models with structured reasoning patterns and tool integration frameworks. This section examines the ReAct agent architecture and the OpenAI Agents SDK, which form the foundation of the Slurm Agent implementation.

2.4.1 ReAct: Reasoning and Acting

The ReAct paradigm, introduced by Yao et al. [48], represents a significant advancement in LLM-based agent design. ReAct synergizes **reasoning** (generating verbal reasoning traces) with **acting** (taking actions that affect the environment), enabling agents to solve complex tasks that require both planning and external interaction.

The Think-Act-Observe Loop

ReAct agents operate through an iterative loop with three distinct phases:

1. **Think:** The agent generates a reasoning trace explaining its current understanding and intended next step. This internal monologue helps decompose complex problems and maintain task focus.
2. **Act:** Based on the reasoning, the agent selects and executes an action—typically invoking an external tool with specific parameters.
3. **Observe:** The agent receives the result of its action from the environment and incorporates this observation into its context for the next iteration.

This cycle repeats until the agent determines it has sufficient information to provide a final response, or encounters a termination condition.

ReAct Agent architecture

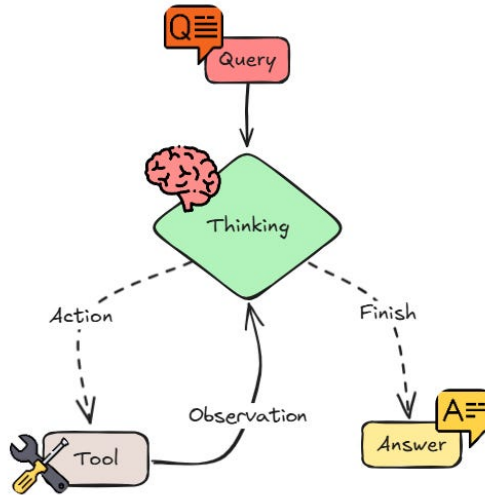


Figure 2.5: The ReAct (Reason + Act) reasoning loop [48]. Starting from a user *Query*, the agent enters an iterative cycle: it *Thinks* (produces a chain-of-thought reasoning trace to determine what information is missing), *Acts* (selects and invokes an external Tool such as a search engine, database, or command executor), and *Observes* (reads the raw result returned by the tool). The agent then returns to the thinking phase to decide whether it has sufficient information or needs another tool call. Once the goal is met the loop exits and the agent emits a final *Answer*. Grounding reasoning in real tool observations at each step suppresses hallucination by preventing the model from fabricating facts that a tool call would contradict. [3]

ReAct is suitable for HPC operations because: (1) Slurm diagnosis requires chaining multiple queries (queue state, priority, partition availability) before concluding; (2) the reasoning trace makes tool selections auditable; (3) observations can revise the agent’s plan when initial assumptions are wrong.

2.4.2 OpenAI Agents SDK

The OpenAI Agents SDK [26] provides the execution framework used in this project. Its key abstractions are:

- **Agent:** An LLM with instructions, tools, and behavioral config. Separate agents isolate read-only observation from state-changing operations.
- **Runner:** Manages the execution loop—sending messages, processing tool calls, collecting results.
- **Tools:** Functions agents invoke to interact with external systems, with automatic parameter validation.
- **Handoffs:** Transfer control between specialized agents, passing conversation context.

The Runner's internal loop naturally implements ReAct: (1) collect messages, (2) send to LLM, (3) if tool calls, execute and loop; (4) if final text, return; (5) if handoff, transfer to target agent. The SDK also integrates with MCP servers, treating MCP tools as native agent tools.

Chapter 3

Related Tools and Works

This chapter reviews work related to natural-language assistance for Slurm-based HPC operation.

3.1 HPC Management Interfaces

Slurm provides expressive CLI tools (`squeue`, `sacct`, `sinfo`, `sbatch`, `scancel`) but assumes scheduler expertise [49, 37]. Portals such as Open OnDemand [18] and JupyterHub [19] reduce barriers but still require manual parameter selection. Monitoring tools (Open XDMoD [28], Prometheus/Grafana [30, 14]) are observational and do not convert requests into validated actions.

3.2 LLM-Based HPC Systems

Recent work integrates LLMs with HPC, though with different goals. Chat AI [11] deploys LLMs as services on Slurm-backed GPUs but does not use LLMs to *manage* the cluster. LangChain-Parsl [2] connects an LLM agent to HPC workflows via the Parsl framework but lacks HITL safety and native Slurm tool integration. Fujitsu’s AI Computing Broker [13] optimizes GPU allocation across clusters at the resource-brokering layer.

Table 3.1: Comparison of LLM-Based HPC Systems

System	NL Mgmt	Tool Calls	HITL	Slurm-Native
Chat AI	No	No	N/A	Yes
LangChain-Parsl	Yes	Yes	No	No (Parsl)
Fujitsu ACB	No	No	N/A	Partial
Slurm Agent	Yes	Yes	Yes	Yes

3.3 LLM Agents and Tool-Augmented Reasoning

ReAct combines reasoning traces with actions [48]. Toolformer demonstrates tool-calling patterns [38]. Agent surveys identify planning, memory, tool use, and multi-agent coordination as key design dimensions [8, 47]. Frameworks such as the OpenAI Agents SDK and AutoGen provide implementation support including tool invocation, streaming, and handoff patterns [26, 46]. These are general-purpose; a Slurm assistant must additionally know which tools are read-only, which require confirmation, and how to interpret scheduler state.

3.4 Protocol-Based Tool Integration

The Model Context Protocol (MCP) defines a standard client-server interface for exposing tools to AI applications [4, 5]. This project applies MCP to a Slurm-specific operational setting with read/write tool separation, mock scenarios for evaluation, and traceable behavior.

3.5 Natural Language to Structured Operations

Semantic parsing (Seq2SQL [53], Spider [50], RAT-SQL [43]) maps language to structured queries. For Slurm, the operation space is a set of scheduler tools with parameters, states, and safety policies rather than a database schema. This makes the system closer to grounded operation generation with action risk management.

3.6 Safety and Evaluation

Alignment methods [6, 27] improve assistant behavior but cannot guarantee correctness in live systems. LLM-as-judge evaluation [52] does not verify tool-grounded workflows. The benchmark in this thesis therefore checks tool selection, routing, HITL compliance, and state transitions across 3,135 Slurm-specific scenarios.

Chapter 4

Proposed Approaches

4.1 Proposed Slurm Agent Overview

The proposed system is a stateful Slurm Agent for translating natural-language operational requests into grounded scheduler interactions. Its purpose is to study how an LLM-based agent can interpret Slurm intent, choose typed tools, separate read-only inspection from state-changing operations, and produce traces that can be evaluated against expected behavior. A key contribution is the domain-adapted fine-tuning of an open-weight model (Qwen2.5-14B-Instruct) using QLoRA, enabling local deployment without reliance on commercial APIs while maintaining tool-calling accuracy through training data aligned exactly with the inference-time prompts and schemas. The user-facing interface is only an entry point; the main design object is the agent control flow, the fine-tuning pipeline, and the evaluation method around it.

4.1.1 System Objective

The system sits between a natural-language request and the Slurm command surface. A request such as “why is my job pending?”, “show failed jobs for this account”, or “cancel job 12345” is not executed as a free-form shell command. Instead, the agent maps the request onto a constrained set of MCP tools, retrieves current or simulated cluster state, and decides whether the task is observational, diagnostic, documentation-oriented, or state-changing.

This objective creates four design requirements. First, the agent must ground claims in Slurm evidence rather than model memory alone. Second, commands that modify cluster state must pass through an explicit confirmation path. Third, the runtime must expose enough trace information to evaluate tool choice, routing, confirmation behavior, and state-transition consistency. Fourth, the system should operate with a locally fine-tuned open-weight model so that HPC sites retain data sovereignty and avoid per-query API costs, while still matching the behavioral quality of commercial models. The evaluation benchmark validates both the architecture itself (via the commercial baseline) and the fine-tuned model’s ability to replace that baseline for local deployment.

The following sections detail the architecture that satisfies these requirements: the dual-agent routing design (Section 4.3), the MCP tool protocol (Section 4.4), the ReAct reasoning cycle (Section 4.5), the fine-tuning pipeline (Section 4.5.3), and the scenario-grid evaluation method (Section 6.1).

4.2 Requirements Overview

This section defines the functional and non-functional requirements that guide system design. Requirements are derived from the project goals stated in Section 4.1 and the operational needs of HPC administrators and researchers.

4.2.1 Functional Requirements

Table 4.1: Functional Requirements

ID	Requirement
FR1	The system shall interpret natural-language Slurm queries, including variations in phrasing and implicit context, and map them to appropriate typed tool calls.
FR2	The system shall retrieve live cluster state (queues, nodes, partitions, accounting, reservations, scheduler diagnostics) via read-only Slurm tools and present results grounded in tool output.
FR3	The system shall execute state-changing operations (submit, cancel, hold, release, requeue, node control, account management) only after explicit user confirmation.
FR4	The system shall retrieve relevant Slurm documentation from a local corpus using hybrid retrieval (lexical + semantic), with web search as a fallback when the local corpus does not cover the topic.
FR5	The system shall maintain multi-turn conversation context, enabling follow-up questions and contextual references within a session.
FR6	The system shall provide real-time feedback during execution through structured status events, making tool selections and intermediate results visible to the user.
FR7	The system shall support deterministic evaluation against mock cluster states, producing behavioral traces that can be compared to expected tool selections, routing decisions, and state transitions.

4.2.2 Non-Functional Requirements

Table 4.2: Non-Functional Requirements

ID	Requirement
NFR1	Safety: The system shall never execute cluster-mutating commands without explicit confirmation. The confirmation mechanism shall be enforced at the tool layer, independent of LLM behavior.
NFR2	Accuracy: Responses shall be grounded in tool output. The system shall not fabricate cluster state, job IDs, node names, or resource figures.
NFR3	Latency: Simple read-only queries shall complete within interactive response times. Long-running workflows shall emit incremental status events.
NFR4	Extensibility: New Slurm tools shall be addable via the MCP server without modifying the agent core. Tool schemas are discovered at runtime.
NFR5	Context Efficiency: The system shall avoid unnecessary growth of LLM context by lazy-loading documentation, capping retrieval output size, and separating large artifacts from conversational memory.
NFR6	Reproducibility: Evaluation runs shall be deterministic given the same mock state, model, and test case. Provider and model shall be configurable per session.

4.2.3 Requirements Traceability

Table 4.3 maps each requirement to the project goals defined in Section 4.1.

Table 4.3: Requirements Traceability to Project Goals

Requirement	G1 <i>NL Slurm Access</i>	G2 <i>Control Flow</i>	G3 <i>MCP Tools</i>	G4 <i>FT & Eval</i>
FR1: Intent Interpretation	✓			✓
FR2: Cluster State Retrieval	✓		✓	
FR3: Confirmed State Changes	✓	✓	✓	
FR4: Documentation Retrieval	✓		✓	
FR5: Multi-Turn Conversation	✓	✓		
FR6: Real-Time Feedback		✓		✓
FR7: Deterministic Evaluation			✓	✓
NFR1: Safety		✓		✓
NFR2: Accuracy	✓			✓
NFR3: Latency	✓			✓
NFR4: Extensibility			✓	
NFR5: Context Efficiency		✓		✓
NFR6: Reproducibility				✓

4.2.4 Constraints

Technology: Python backend, MCP-based tool layer, local Ollama models for reproducibility with optional remote providers for development and comparison.

Environment: Evaluation uses mock Slurm states exposed by the MCP server. Real deployment assumes standard Slurm command availability and site permissions.

Scope: The system targets broad Slurm assistance (monitoring, diagnosis, submission, job control, accounting, documentation). High-risk operations are gated by explicit confirmation rather than unrestricted automation.

4.3 System Architecture

This section presents the overall architecture of the Slurm Agent system, describing the major components, their interactions, and the rationale behind architectural decisions. The design uses common web and agent frameworks, but the architectural contribution is the Slurm-specific control plane built on top of them: broad Slurm tool coverage, read/write separation with confirmation, and local documentation grounding.

4.3.1 Architecture Overview

The Slurm Agent is organized as a four-layer stack. At the top, a thin **Interface Layer** accepts natural-language requests and relays confirmation decisions without embedding any domain

logic. Below it, the **Agent Layer** houses the Observer/Operator multi-agent backend that performs ReAct reasoning, retrieves documentation, and enforces safety routing—the Observer handles read-only inspection and decides when to hand off to the Operator for state-changing operations. The **Protocol Layer** exposes each Slurm operation as a typed MCP tool with parameter validation, isolating the agent from raw command-line construction. At the bottom, the **Infrastructure Layer** provides the actual execution targets—either a live Slurm cluster or a mock scenario engine—alongside the local documentation corpus and external services.

This separation ensures that the boundary between reading cluster state and mutating it is an explicit architectural seam rather than an implicit convention. Read-only inspection flows entirely within the Observer path, while any state-changing operation must cross into the Operator path and pass through a confirmation gate before reaching the MCP tool layer.

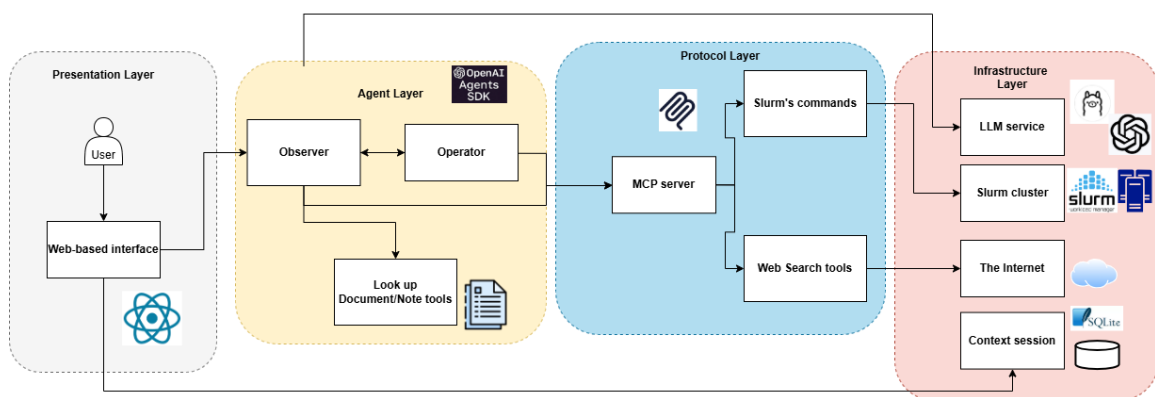


Figure 4.1: Slurm Agent System Architecture. The system is organised into four layers. (1) **Presentation Layer**: a React-based web interface through which the user submits natural-language requests. (2) **Agent Layer**: the reasoning core built on the OpenAI Agents SDK, comprising an Observer that interprets user intent and gathers context, an Operator that decides on actions and coordinates tool use, and a knowledge-base lookup tool for cluster documentation and notes. (3) **Protocol Layer**: an MCP (Model Context Protocol) server that translates agent decisions into typed, validated calls to Slurm commands (`squeue`, `scancel`, etc.) and web-search tools, decoupling agent logic from execution details. (4) **Infrastructure Layer**: the execution targets, including the LLM service (local fine-tuned model or cloud API), the Slurm cluster, the internet for external lookups, and a SQLite context store that persists session state across turns.

4.3.2 Design Rationale

The main design decision is to treat Slurm interaction as a controlled operation surface rather than as open-ended command generation. A general chatbot can explain commands, but a cluster assistant must also decide when to inspect live state, when to consult documentation, when to ask for clarification, and when to refuse or delay execution until a user confirms intent. The architecture therefore combines two forms of grounding:

- **Operational grounding**: current cluster facts come from Slurm tools exposed through MCP.

- **Documentation grounding:** command syntax, reason-code meanings, and policy-like explanations come from local Slurm documentation retrieval.

The architecture is also designed to support incomplete or ambiguous user requests. When the user says a phrase such as “cancel that job”, the system must determine whether a specific job is already known from context. If the target is unclear, the safe behavior is to ask for clarification. If the target is clear but the action changes cluster state, the safe behavior is to create a pending action and require confirmation. This is why session state and pending-action persistence are architectural components rather than UI conveniences.

4.3.3 Component Descriptions

Interface Layer

The Interface Layer is intentionally thin. It provides an entry point for manual prompts, automated evaluation prompts, and explicit confirmation decisions. This layer is responsible for:

- forwarding natural-language requests to the agent backend,
- presenting final responses and selected status events,
- relaying confirm/cancel decisions for pending actions,
- exposing traces needed for debugging and evaluation.

The interface is therefore support infrastructure, not the central contribution. Agent behavior is defined by the backend control flow and can be exercised either manually or through the automated evaluator.

Agent Layer

The Agent Layer implements core reasoning and orchestration using a dual-agent architecture. Both agents share the same fine-tuned LLM and MCP tool connection but operate under distinct system prompts and tool-access policies:

- **Observer:** The primary entry point for every request. It implements the ReAct reasoning loop (Think → Act → Observe), classifies user intent, and handles all read-only operations—queue inspection, node status, accounting queries, reservation checks, scheduler diagnostics, documentation retrieval, and web search. When the request requires cluster state modification, the Observer produces a structured handoff directive specifying the action, target scope, and tool, then delegates to the Operator.
- **Operator:** Receives handoff directives for state-changing operations such as job submission, cancellation, hold, release, requeue, and configuration updates. It may perform a single discovery read to resolve targets (e.g., listing jobs before cancelling), then invokes

the action tool. Execution is gated by a human-in-the-loop confirmation step for all destructive operations.

This division is central because incorrect execution can waste compute time, interrupt other users, or change shared cluster policy. The Observer never holds action tools, and the Operator never runs without an explicit handoff—ensuring that the boundary between reading and mutating state is enforced by the framework rather than by prompt compliance alone.

Beyond safety enforcement, the dual-agent design provides a measurable **tool-scope reduction** benefit for capacity-constrained local models. The full Slurm tool surface comprises 18 distinct tools; presenting all of them simultaneously to a 14B-parameter model degrades tool-selection accuracy as the model’s attention must discriminate among semantically similar options (e.g., `sacct` vs. `squeue` vs. `sacctmgr_list`). By partitioning tools into two focused subsets—approximately 10 read-only tools for the Observer and approximately 8 action tools for the Operator—each agent operates within a smaller selection space where correct tool identification is easier. Ablation experiments in Section 6.2.9 quantify this effect.

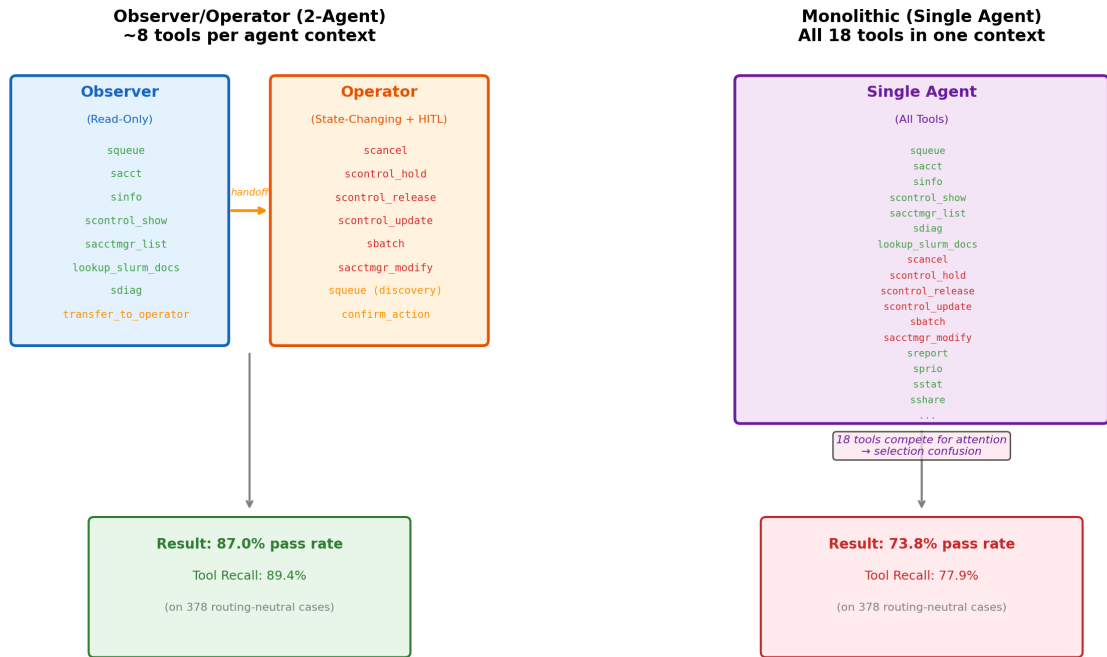


Figure 4.2: Tool partitioning comparison: Observer/Operator split (left) versus Monolithic configuration (right). In the split design, the Observer receives only read-only tools (green) plus the handoff mechanism (orange), while the Operator receives only destructive/action tools (red) and a small set of discovery reads. The Monolithic baseline exposes all 18 tools to a single agent context. Reducing the per-agent tool count from 18 to approximately 8 narrows the selection space and directly improves tool-call accuracy for the 14B-parameter model, as quantified by the ablation experiments in Section 6.2.9.

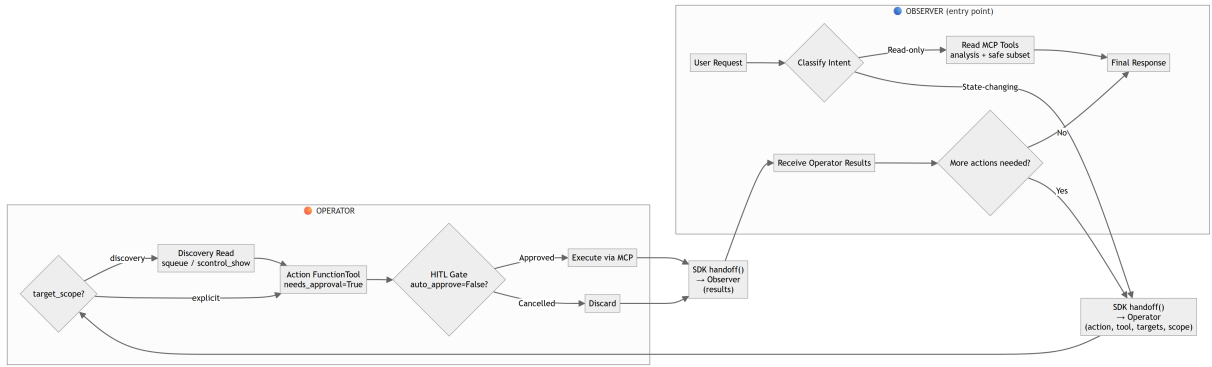


Figure 4.3: Observer/Operator routing and confirmation flow. Every request enters the Observer, which classifies intent and either handles read-only operations directly (via Read MCP Tools) or issues a `transfer_to_operator` directive carrying the action, tool, targets, and scope for state-changing requests. The Operator resolves the target scope—performing a discovery read if needed—then invokes the action tool. All destructive tool calls pass through a HITL confirmation gate: approved actions execute and results are returned to the Observer via `transfer_to_observer`; cancelled actions are discarded. The Observer then decides whether additional tool calls are required or synthesises the final response.

Protocol Layer

The MCP Protocol Layer wraps each Slurm command as a typed tool with parameter validation, operational semantics preservation, safe subprocess execution, and structured output formatting. It decouples agent logic from Slurm implementation details and provides a security boundary for command execution.

Infrastructure Layer

The Infrastructure Layer provides external execution targets and supporting services. It includes: (1) the LLM provider (local Ollama, self-hosted fine-tuned model via OpenAI-compatible endpoint, or cloud API), (2) the Slurm cluster itself (real CLI access or mock scenario engine for evaluation), (3) a local documentation corpus indexed for retrieval, (4) a web search service for fallback queries, and (5) a SQLite context database for session persistence.

4.3.4 Data Flow and Deployment

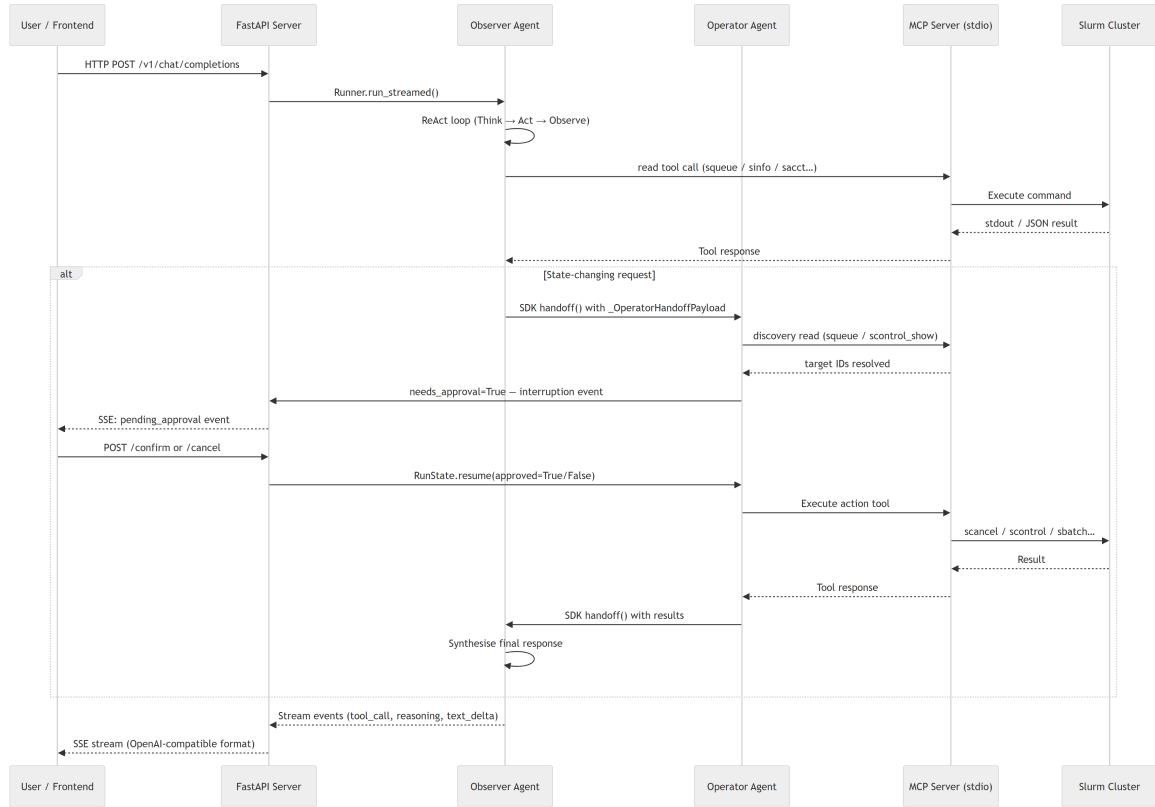


Figure 4.4: Request lifecycle data flow (sequence diagram). A user request arrives at the FastAPI server over HTTP or WebSocket. The server invokes the ReAct Agent, which classifies intent via the Router and delegates to the appropriate Observer or Operator sub-agent. The sub-agent communicates with the MCP Server over stdio, which in turn executes the relevant Slurm command against the cluster. Results propagate back up the call chain; the agent synthesises a response with embedded trace events that are streamed to the client as a Server-Sent Events (SSE) stream. Dangerous operations trigger a confirmation interruption between the sub-agent and the MCP Server before execution proceeds.

4.4 Module Specifications

This section specifies the operational workflow of each functional module. Each module is described by its role, processing stages, and interaction contracts.

4.4.1 Observer Module

The Observer is the primary entry point for all requests and handles read-only operations. It follows a ReAct loop: (1) classify user intent, (2) select and invoke read-only MCP tools, (3) chain additional tool calls if evidence is insufficient, (4) synthesize a grounded response. If a state-changing operation is detected, the Observer produces a structured handoff containing the classified intent, gathered evidence, and target identifiers, then transfers control to the Operator.

The Observer accesses read-only tools spanning queue inspection (squeue), node status (sinfo), job history (sacct), scheduler diagnostics (sdiag, sprio), resource statistics (sstat, sreport), configuration display (scontrol_show), documentation (lookup_slurm_docs), and web search. At runtime, the Observer sees approximately 10 tools from the agent’s 18-tool surface; the full MCP server exposes 67 tool endpoints including variants and utilities.

4.4.2 Operator Module

The Operator manages all state-changing operations. It is activated exclusively through Observer handoffs and enforces a Human-in-the-Loop (HITL) confirmation protocol before any destructive action.

Workflow: (1) Receive handoff with classified intent and targets. (2) Optionally verify current state with a discovery read. (3) Present the planned action to the user via a pending action event. (4) Block until explicit user confirmation or cancellation. (5) Execute the tool call upon confirmation. (6) Perform post-execution verification and return results.

Tool Classification: Tools are dynamically classified at startup into three categories—*analysis* (read-only, no confirmation needed), *safe* (information retrieval), and *dangerous* (state-modifying, always requires HITL). Any tool whose name or description implies state modification is classified as dangerous.

4.4.3 MCP Tool Layer

The MCP server provides 67 typed tools organized into functional groups. Table 4.4 shows the distribution across operational categories. The grouping reflects Slurm’s own command families: read-only monitoring tools dominate (suitable for the Observer), while job control, node management, and accounting tools require Operator-level confirmation before execution.

Table 4.4: MCP Tool Groups

Group	Count	Examples
Queue/Monitoring	10	squeue, sinfo, sacct, sdiag
Job Control	8	sbatch, scancel, scontrol_hold
Node Management	6	scontrol_node, power up/down
Reservations	4	create/delete/update reservation
Accounting	8	sacctmgr show/add/modify/delete
Cluster Admin	5	reconfigure, shutdown
Documentation & Web	3	lookup_slurm_docs, web_search
Other (triggers, charts, mock)	23	triggers, chart generators, test utilities
Total	67	

Dual Execution Mode: The server supports *real mode* (translates tool calls to Slurm CLI commands) and *mock mode* (operates on mutable in-memory state with five predefined scenarios). Mock mode enables deterministic evaluation from known starting states.

Parameter Validation: Before execution, the MCP layer validates job IDs, node names, partition names, and scans batch scripts for dangerous patterns, providing a safety layer independent of LLM reasoning quality.

4.4.4 Documentation Retrieval

A local corpus of 273 official Slurm documentation pages enables grounded answers without relying on LLM parametric memory. The `lookup_slurm_docs` tool performs keyword-based lookup against an indexed corpus. When local docs are insufficient, the agent falls back to `web_search` and `fetch_web_content` for external sources.

4.4.5 Evaluation Pipeline

Each benchmark test case specifies: a natural-language request, source cluster state, target state, expected tools, routing decision, and HITL requirement. Scoring uses five weighted dimensions:

$$\text{Overall} = 0.30 \cdot \text{ToolRecall} + 0.25 \cdot \text{Routing} + 0.25 \cdot \text{HITL} + 0.10 \cdot \text{Keywords} + 0.10 \cdot \text{State} \quad (4.1)$$

A case passes when $\text{overall} \geq 0.80$. Aggregate metrics include pass rate, behavioral agreement rate, and safety violation rate. An optional LLM-as-judge provides supplementary 1–5 quality scores without affecting pass/fail determination.

4.5 Large Language Model Architecture

4.5.1 ReAct Reasoning Loop

The agent follows the ReAct paradigm [48], alternating reasoning traces with tool actions:

$$(t_1, a_1, o_1), (t_2, a_2, o_2), \dots, (t_n, a_n, o_n), r \quad (4.2)$$

where each thought t_i conditions on $(q, t_1, a_1, o_1, \dots, t_{i-1}, a_{i-1}, o_{i-1})$. This suits HPC operations because Slurm diagnosis typically requires multiple tool calls, and the visible reasoning trace makes decisions auditable.

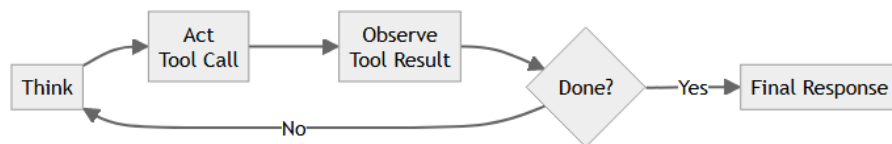


Figure 4.5: ReAct reasoning cycle. The agent alternates between three phases: *Think* (produce a reasoning trace conditioned on the conversation history), *Act* (invoke a tool call with structured arguments), and *Observe* (receive and parse the tool response). The cycle repeats until the agent determines the request is resolved, at which point it emits a *Final Response*. This loop structure makes intermediate reasoning steps auditable and naturally handles multi-step Slurm operations that require chaining several tool calls.

4.5.2 Instruction Design

The **Observer** prompt defines tool categories, routing rules for Operator handoff, evidence requirements (responses must cite tool output), documentation retrieval triggers, and output structure.

The **Operator** prompt defines confirmation requirements for destructive operations, scope preservation rules, discovery-read permissions, error handling, and return protocol.

Both agents share the same underlying model and MCP connection but receive strictly different tool lists at runtime—the Observer sees only read-only tools plus the handoff mechanism, while the Operator sees only action tools and verification reads. This partitioning serves a dual purpose: it enforces safety boundaries by making it structurally impossible for the Observer to invoke destructive tools, and it reduces the per-agent tool count from 18 to approximately 8, directly addressing the tool-selection degradation observed in 14B-parameter models when presented with large tool surfaces. The system supports multiple LLM backends (local Ollama, OpenAI API, or a self-hosted fine-tuned model behind an OpenAI-compatible endpoint) with identical prompts across all providers, so behavioral differences reflect model capability rather than configuration drift.

4.5.3 Domain-Adapted Fine-Tuning

While the system achieves strong results with commercial API models, reliance on external providers introduces latency, cost, and privacy concerns for production HPC environments. To address this, the project performs domain-adapted fine-tuning of an open-weight model (Qwen2.5-14B-Instruct) as the local agent backbone, using knowledge distilled from a capable teacher model (GPT-5-mini) through a controlled MCP mock environment.

MCP Mock Environment for Data Collection

Training an agent requires diverse, realistic tool interactions—but running against a live Slurm cluster is impractical for data generation at scale. The solution is a *mock MCP server* that simulates the full tool surface with deterministic, resettable cluster states.

The mock implements every MCP tool (queue queries, node status, accounting, job submission, cancellation, etc.) backed by in-memory state that can be configured per scenario. Each evaluation benchmark case defines an initial cluster snapshot—jobs in various states, node configurations, queue policies—and the expected agent behavior against that snapshot. This allows thousands of agent interactions to be collected without access to real hardware.

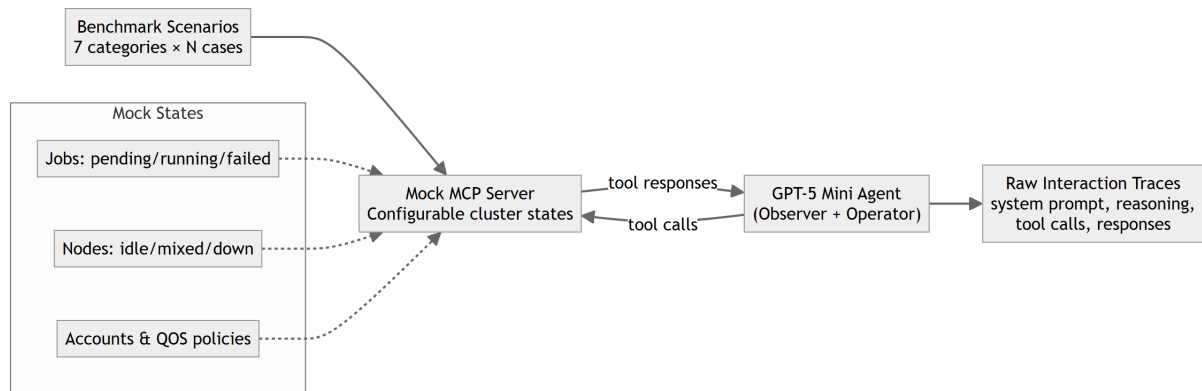


Figure 4.6: MCP mock environment for training data collection. Benchmark scenarios across 11 categories are fed to a GPT-5-mini agent running via the same Observer/Operator pipeline used at inference time. The agent interacts with a configurable mock MCP server whose in-memory cluster state covers jobs (pending, running, failed), nodes (idle, mixed, down), and account/QOS policies. The full exchange—system prompt, reasoning steps, tool calls, tool responses, handoff directives, and final answers—is captured as a raw interaction trace, which forms the distillation corpus for fine-tuning.

Distillation from GPT-5-mini

The training data is generated by running GPT-5-mini as the agent backbone against the mock MCP server across the full benchmark suite. Each successful run produces a complete interaction trace: the system prompt, user request, the model’s reasoning steps, tool calls with arguments, tool responses, handoff directives, and final answers.

This distillation approach offers two advantages over manual annotation. First, the teacher model demonstrates *correct behavioral patterns*—proper tool selection, evidence-grounded responses, appropriate handoff decisions—that would be expensive to write by hand for hundreds of scenarios. Second, the traces are guaranteed to be *format-consistent* with the inference-time protocol because they are produced by the same runtime pipeline that the student model will later serve.

The pipeline processes each passing trace into the student model’s chat template format: a system prompt, alternating user/assistant/tool turns, with tool calls encoded as structured JSON matching the exact schema the model must generate at inference time.

Data Augmentation

Raw distillation traces are augmented through several strategies to increase diversity and prevent overfitting:

1. **Handoff splitting.** Traces involving an Observer-to-Operator handoff are split at the handoff boundary into two independent training samples. The Observer sample ends with the handoff directive; the Operator sample begins from the handoff context. This doubles the effective sample count for multi-agent flows.
2. **Role-scoped tool filtering.** Each sample carries only the tools available to its active role—Observer samples include read-only tools plus the handoff mechanism; Operator samples include only action tools and discovery reads. This teaches the model that tool availability is conditional on role, preventing cross-role hallucination (e.g., an Observer invoking `scontrol_hold`).
3. **Prompt variation.** User queries are rephrased with minor lexical and structural changes (e.g., “cancel job 4021” → “kill job 4021”, “why is my job stuck?” → “job 5523 has been pending for hours, what’s wrong?”) to reduce surface-form memorization while preserving intent semantics.
4. **Scenario diversity.** The benchmark covers 11 categories (read, diagnose, action, bulk, safety, submission, multi_step, account, edge, docs, domain) with multiple scenarios per category, ensuring the model sees the full operational surface rather than a narrow slice.

The resulting dataset contains 3,329 samples after role splitting (2,625 unique after deduplication) and 615 held-out test samples, stratified 80/20 by category and scenario to ensure evaluation coverage mirrors training distribution.

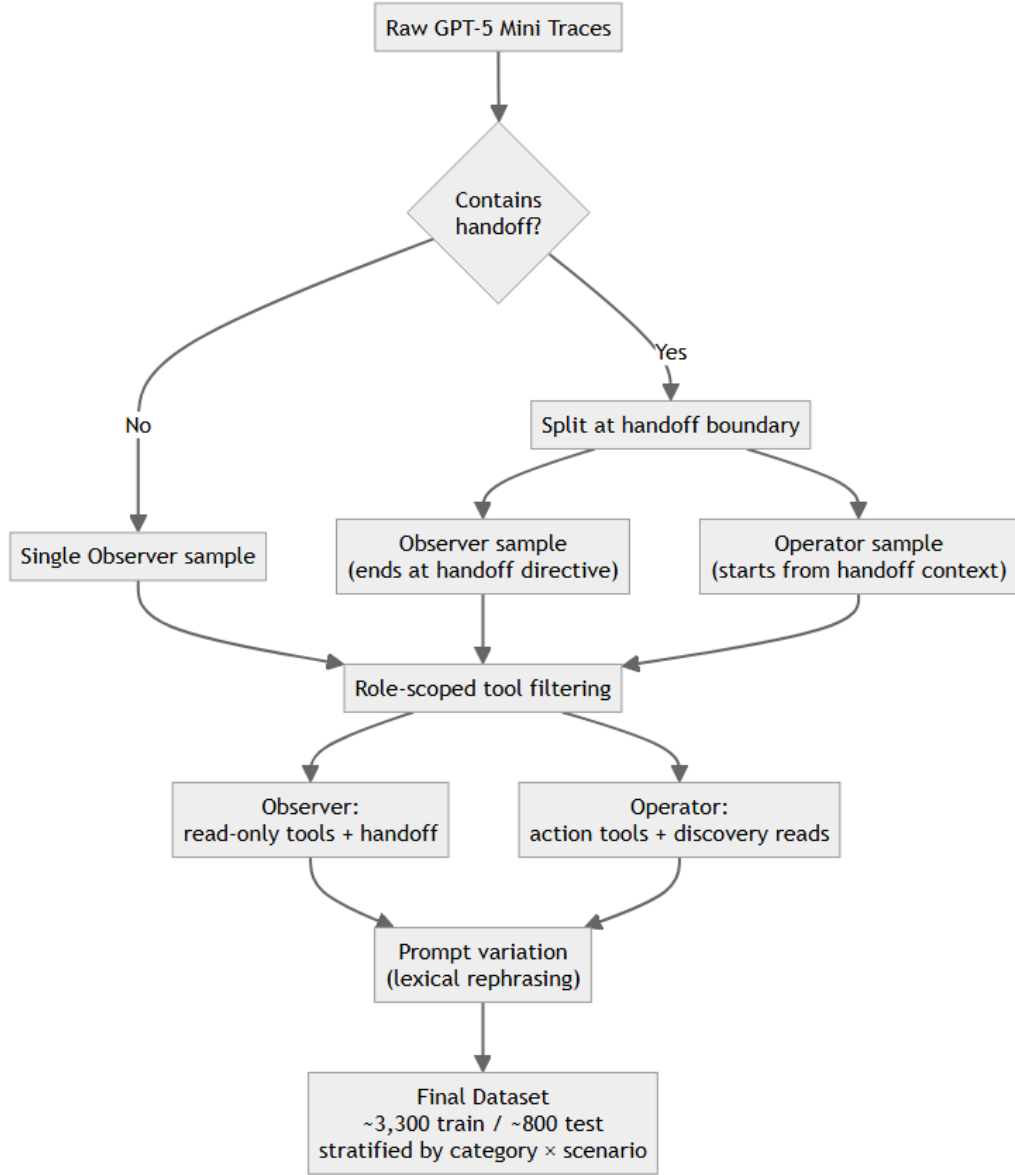


Figure 4.7: Training data augmentation pipeline. Raw GPT-5-mini traces are first classified by whether they contain an Observer-to-Operator handoff. Traces with a handoff are split at the boundary into two independent samples (an Observer sample ending at the handoff directive and an Operator sample starting from the handoff context); traces without a handoff produce a single Observer sample. All samples then pass through role-scoped tool filtering, which retains only the tools available to the active role—read-only tools plus the handoff mechanism for the Observer, action tools and discovery reads for the Operator. Finally, prompt variation (lexical rephrasing of user queries) is applied to reduce surface-form memorisation. The resulting dataset contains 3,329 training samples (2,625 unique after deduplication) and 615 held-out test samples, stratified by category and scenario.

Training Configuration

The fine-tuning uses QLoRA [10], a parameter-efficient method that quantizes the frozen base model to 4-bit precision and trains a small low-rank adapter on top. The LoRA adapter is applied

to all 48 transformer layers rather than only the upper layers—preliminary experiments with partial-layer adaptation (layers 36–47) showed base-model bleed-through where the pretrained model’s CLI-style tool invocation prior overrode the adapter’s structured JSON output on certain prompt categories.

The adapter is trained with a standard causal language modeling objective, but the loss is masked so that only assistant-generated tokens (tool calls and responses) receive gradient signal. System prompts, user messages, and tool-response observations are treated as context-only. Each sample’s tool definitions are embedded in the input exactly as they would appear at inference time, ensuring the model learns to condition its generation on the available tool surface.

Chapter 5

Implementation

5.1 Agent Backend Implementation

The backend maintains session-aware multi-agent execution, exposes an OpenAI-compatible chat API, and enforces confirmation workflows for state-changing operations.

5.1.1 Technology Stack

- **FastAPI + Uvicorn:** API layer for chat, health, and session endpoints
- **OpenAI Agents SDK:** multi-agent orchestration and tool-driven execution
- **MCP over SSE:** tool discovery and execution against the Slurm MCP server
- **SQLiteSession:** persistent per-session context and chat history

5.1.2 Agent Topology

The system implements a two-agent architecture with bidirectional handoffs. Both agents share the same LLM instance but differ in tool access and generation policy.

Listing 5.1: Observer agent construction

```
1 observer = Agent(  
2     name="Observer",  
3     instructions=observer_instructions,  
4     model=reasoning_model,  
5     model_settings=ModelSettings(tool_choice="auto"),  
6     mcp_servers=[mcp_observer], # read-only Slurm tools  
7     tools=[slurm_docs_lookup, skill_lookup],  
8     handoffs=[transfer_to_operator],  
9 )
```

Listing 5.2: Operator agent construction

```
1 operator = Agent(  
2     name="Operator",  
3     instructions=operator_instructions,  
4     model=reasoning_model,  
5     model_settings=ModelSettings(tool_choice="required"),  
6     mcp_servers=[mcp_operator], # action Slurm tools  
7     tools=[*dangerous_fns, *operator_read_fns],
```

```
8     handoffs=[transfer_to_observer],  
9 )
```

The Observer uses `tool_choice="auto"` (the model decides whether to call a tool or respond directly), while the Operator uses `tool_choice="required"` (every turn must invoke a tool—either an action or the handoff back). The MCP server instances expose strictly partitioned tool sets: the Observer sees only read-only commands (`squeue`, `sinfo`, `sacct`, `scontrol show`), while the Operator receives action tools (`scancel`, `scontrol update`, `sbatch`) plus a small subset of read tools for target discovery before acting.

5.1.3 Safety and Confirmation Pipeline

Dangerous operations follow a queued-action workflow: (1) the request is transformed into a pending action (not executed), (2) the user sees the action and must explicitly confirm, (3) execution occurs only via `confirm_action`. This two-step control reduces accidental cluster mutations.

5.1.4 Hybrid RAG Documentation Retrieval

The agent includes a local retrieval tool (`lookup_slurm_docs`) that provides Retrieval-Augmented Generation over the official Slurm documentation corpus. The pipeline operates without an external vector database—all indexing is in-memory.

Corpus Preparation

A crawl script fetches all HTML pages from `slurm.schedmd.com`, converts them to Markdown with YAML frontmatter (source URL, title, fetch timestamp), and stores them in a local corpus directory. At the time of writing the corpus contains 273 documents covering commands, configuration, accounting, scheduling, and troubleshooting.

Chunking Strategy

Each document is split at heading boundaries (`#`, `##`, `###`). Chunks smaller than 80 characters are discarded; chunks exceeding 1,400 characters are further split at paragraph boundaries. This produces 2,276 chunks with an average length of approximately 600 characters, balancing retrieval granularity against context window cost.

Dual Retrieval

At query time, the tool runs two parallel scoring passes:

1. **Lexical scoring:** token-overlap BM25-style matching between the normalized query and chunk content, with bonuses for phrase matches and path/title matches.

2. **Semantic scoring:** the query is encoded using the all-MiniLM-L6-v2 sentence-transformer model [33] (384-dimensional embeddings, running on GPU). Cosine similarity against pre-computed chunk embeddings identifies semantically related passages.

Reciprocal Rank Fusion

The two ranked lists are fused using RRF with $k = 60$ [9]. The top-5 results are returned as context snippets (each capped at 950 characters), yielding approximately 1,200 tokens per tool call—a small fraction of the model’s context window.

Pre-computed Embeddings

To avoid a GPU-intensive embedding pass at server startup, a build script (`build_embeddings.py`) pre-computes all 2,276 chunk embeddings and saves them as a compressed NumPy archive (`embeddings.npz`, 3.2 MB). At runtime the agent loads this file in under 100 ms. If the corpus is updated and the embedding count diverges, the system falls back to runtime embedding.

Design Rationale

This hybrid approach addresses a practical limitation observed during evaluation: purely lexical retrieval fails on paraphrased queries (e.g., “how to chain jobs” does not lexically match “dependency afterok”), while purely semantic retrieval can miss exact command-syntax matches. RRF combines both strengths without requiring score calibration. The tool is invoked lazily—only when the agent encounters a documentation question—so it adds zero tokens to non-documentation conversations.

5.1.5 Runtime Services

The backend streams responses using OpenAI-style Server-Sent Events with typed delta fields (`status_update`, `tool_output`, `reasoning_content`, `content`, `chart_artifact`, `pending_actions`). Chart artifacts are filtered from future LLM context to prevent token pollution.

Each chat ID maps to a persistent session storing conversation memory, pending actions, chart context, and runtime config. Evaluation runs use deterministic isolated session IDs for reproducibility.

5.1.6 Fine-Tuned Model Serving

To evaluate the domain-adapted open-weight model under the same agent framework, a lightweight inference server exposes the fine-tuned model through an OpenAI-compatible chat completions endpoint. The server loads the quantized base model with the LoRA adapter merged at startup, and processes requests identically to the commercial API from the agent’s perspective—the same tool-calling format, streaming protocol, and system prompts are used regardless of which backend is active.

As an additional safety layer, the server validates each generated tool call against the `tools` list provided in the request. Any tool call referencing a function not declared in the current request is silently filtered, preventing residual cross-role hallucinations from reaching the agent runtime. This defense-in-depth approach means that even if the model occasionally generates an out-of-scope tool call, it is caught before execution rather than causing a runtime error.

5.2 MCP Server Implementation

The MCP server bridges the AI agents and Slurm infrastructure, translating tool calls into command executions. It is implemented in Python using the official MCP SDK with asyncio-based non-blocking I/O.

5.2.1 Design Principles

Stateless tools: Each tool takes parameters, executes a command, and returns results without maintaining session state between invocations.

Injection prevention: Parameters are never interpolated into shell strings; they are passed as discrete subprocess arguments, preventing command injection regardless of parameter content.

Consistent output: Explicit format flags (`--format`, `--parsable2`) produce pipe-delimited or tabular output suitable for LLM consumption. Empty results return header-only responses.

5.2.2 Tool Surface

Table 5.1 summarizes the 67 tools grouped by operational role.

Table 5.1: MCP Server Tool Reference

Tool Family	Category	Description
Queue/Job inspection	Query	Current queue, job details, accounting history
Node/Partition inspection	Query	Partition state, node reasons, topology
Accounting/Policy	Query	Accounts, QoS, fair-share, priority
Scheduler diagnostics	Query	Controller ping, backfill health, counters
Reports/Licenses	Query	Utilization reports, license pools
Submission	Action	Batch/interactive submission, file staging
Job control	Action	Cancel, hold, release, requeue, suspend
Node control	Action	Drain, resume, power state, GRES updates
Reservation control	Action	Create, update, delete reservations
Controller admin	Admin	Reconfigure, shutdown, debug, tokens
Accounting admin	Admin	Add/modify/delete accounts, archive
Triggers	Admin	List, create, clear event triggers
Documentation	Reference	Local Slurm docs lookup
Web search	Reference	DuckDuckGo search + page fetch

5.2.3 Dual Execution Modes

Real mode: Tool calls invoke actual Slurm CLI commands on the management node with subprocess timeout protection (30s default).

Mock mode: Tools operate against mutable in-memory state simulating a Slurm cluster. Five scenarios (healthy, failed, pending, mixed, debug_needed) provide reproducible starting states. The evaluator resets state before each test case and verifies source-to-target transitions.

5.2.4 Web Search

The `web_search` tool uses DuckDuckGo’s HTML search to retrieve external documentation. The `fetch_web_content` tool retrieves full page text by stripping HTML to readable content (max 4000 chars). These complement the local documentation corpus for topics not covered offline.

5.2.5 Transport

The server uses SSE (Server-Sent Events) transport for remote connections, enabling the agent backend to connect over HTTP. This supports deployment where the MCP server runs on a cluster head node while the agent runs on a separate application server.

5.3 Runtime Interface and Evaluation Integration

The user interface is not treated as a primary research contribution of this thesis. It is a supporting layer that exposes the agent backend, human-confirmation flow, and evaluation runner during development. The main implementation focus remains the agent architecture, MCP tool layer, resettable mock states, dataset construction process, and trace-based evaluation.

5.3.1 Interface Role in the System

The implementation provides a lightweight React + Vite interface for sending natural-language requests to the FastAPI agent service. Its role is deliberately narrow:

- submit user prompts to the OpenAI-compatible chat endpoint,
- display final answers and selected status events from the agent runtime,
- relay explicit confirm/cancel decisions for pending state-changing operations,
- expose trace information that is useful for debugging agent behavior.

The screenshots below are captured from a live session running the fine-tuned Qwen2.5-14B model with the trained QLoRA adapter (DanhVuiVe/slurm-agent-qwen14b-lora-final), demonstrating real agent behaviour produced by the fine-tuned model rather than a base or prompted baseline.

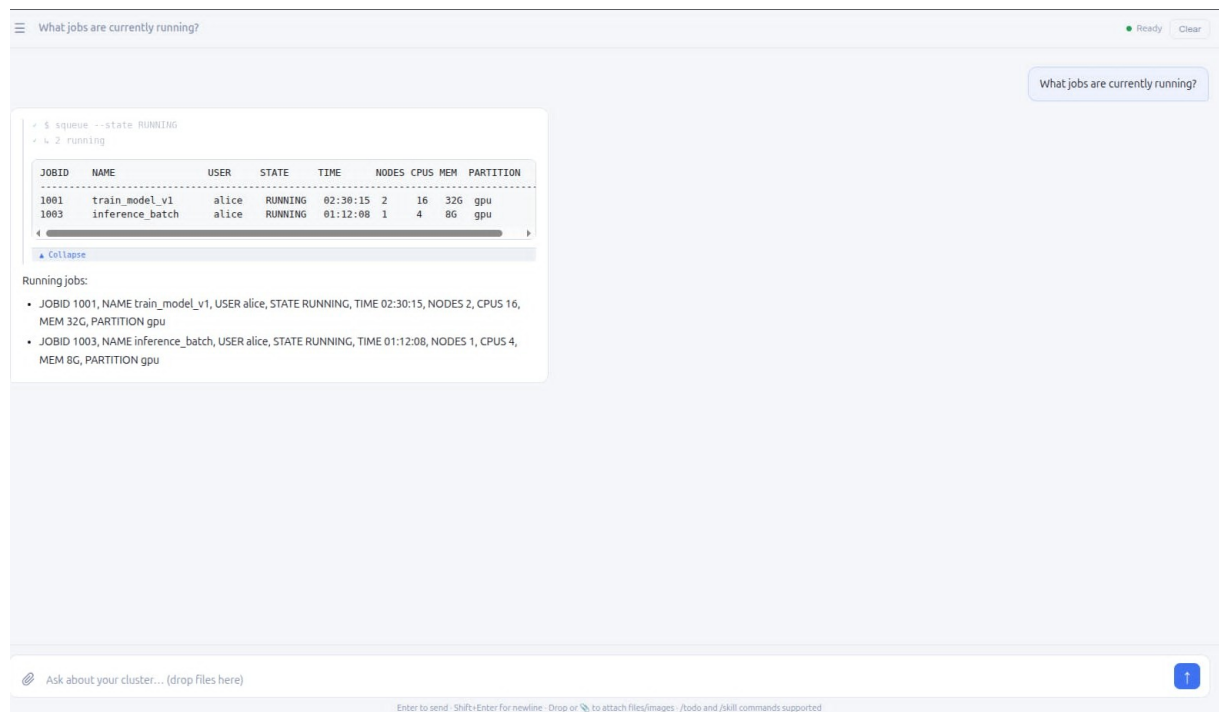


Figure 5.1: HITL Flow (1/3) — Read-only query: the user asks “What jobs are currently running?” The fine-tuned QLoRA model calls `squeue` and renders the result as a structured table plus bulleted breakdown, showing two active jobs (`train_model_v1`, `inference_batch`) for user `alice` with GPU resource details.

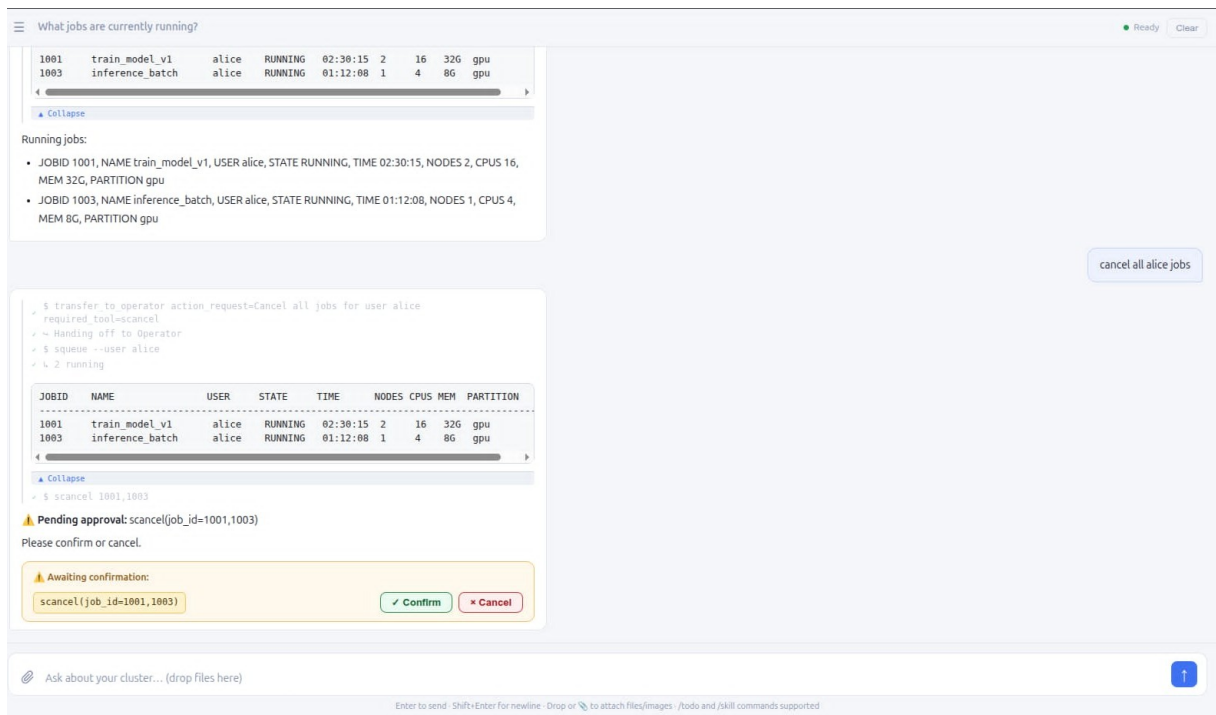


Figure 5.2: HITL Flow (2/3) — Confirmation intercept: the user issues “cancel all alice jobs.” The HITL safety layer intercepts the destructive `scancel` call and presents the exact arguments (`job_id=1001,1003`) with explicit *Confirm* / *Cancel* buttons before any state change is made.

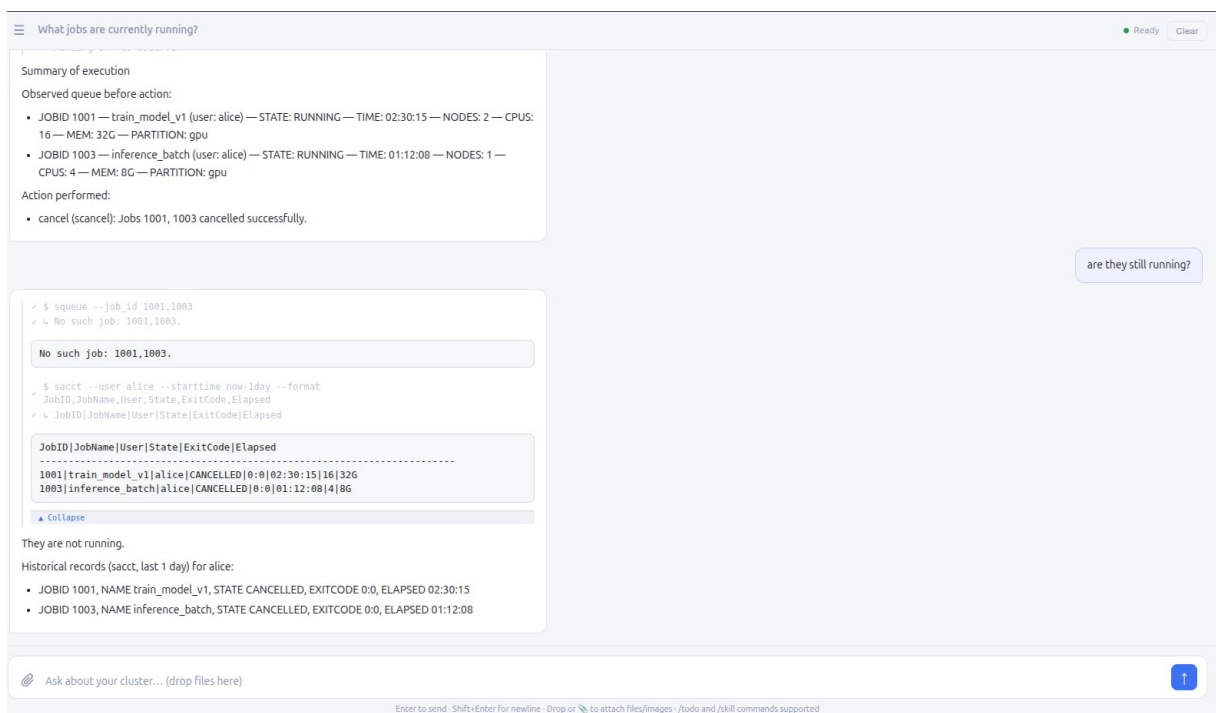


Figure 5.3: HITL Flow (3/3) — Post-execution verification: after user confirmation, `scancel` executes. A follow-up `squeue` returns “No such job” for both IDs; the historical table confirms CANCELLED state with exit codes and elapsed time. All three screenshots are from a live session powered by the `DanhVuiVe/slurm-agent-qwen14b-lora-final` QLoRA adapter.

This interface makes manual testing convenient, but the architectural boundary is the back-

end API.

5.3.2 Additional Capability Screenshots

Beyond the HITL flow, the interface also exposes the Observer’s knowledge-retrieval capabilities. The following figures illustrate web search integration and documentation lookup — two tools that ground agent responses in external sources rather than model memory.

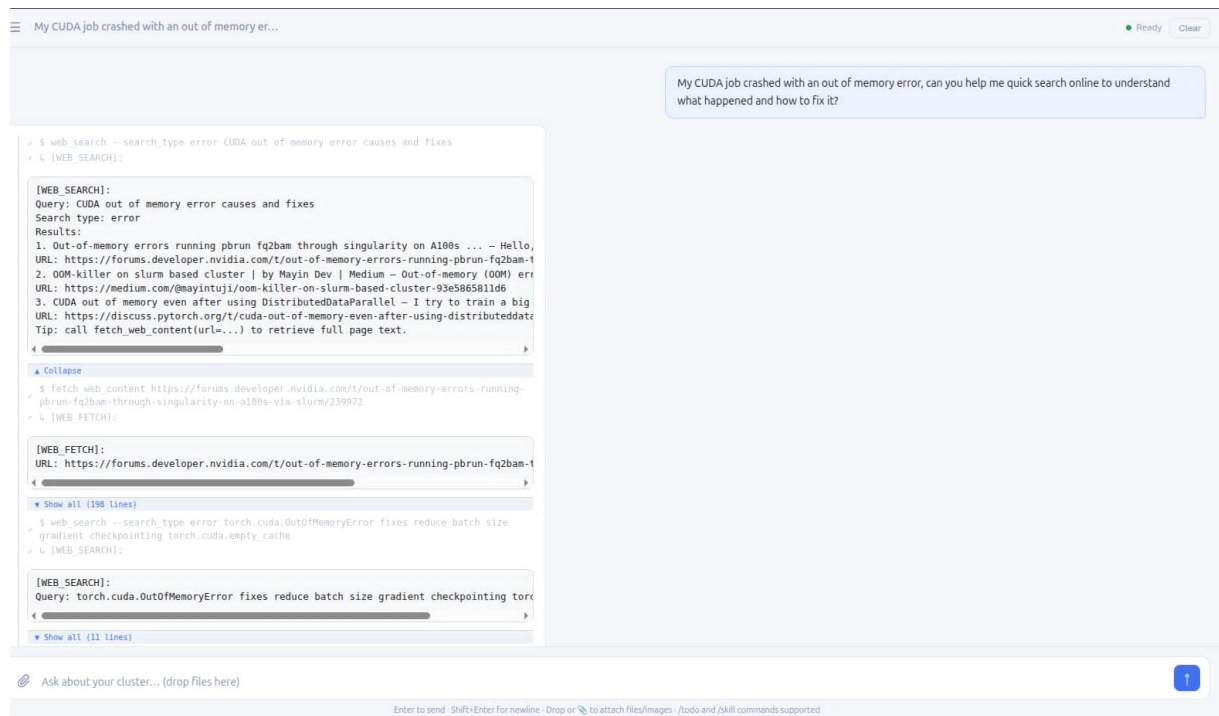


Figure 5.4: Web Search (1/2) — Tool execution phase: the user asks about a CUDA out-of-memory crash. The Observer issues two sequential `web_search` calls followed by a `web_fetch` on a specific NVIDIA developer thread, self-correcting its query after identifying `DistributedDataParallel` as a relevant term.

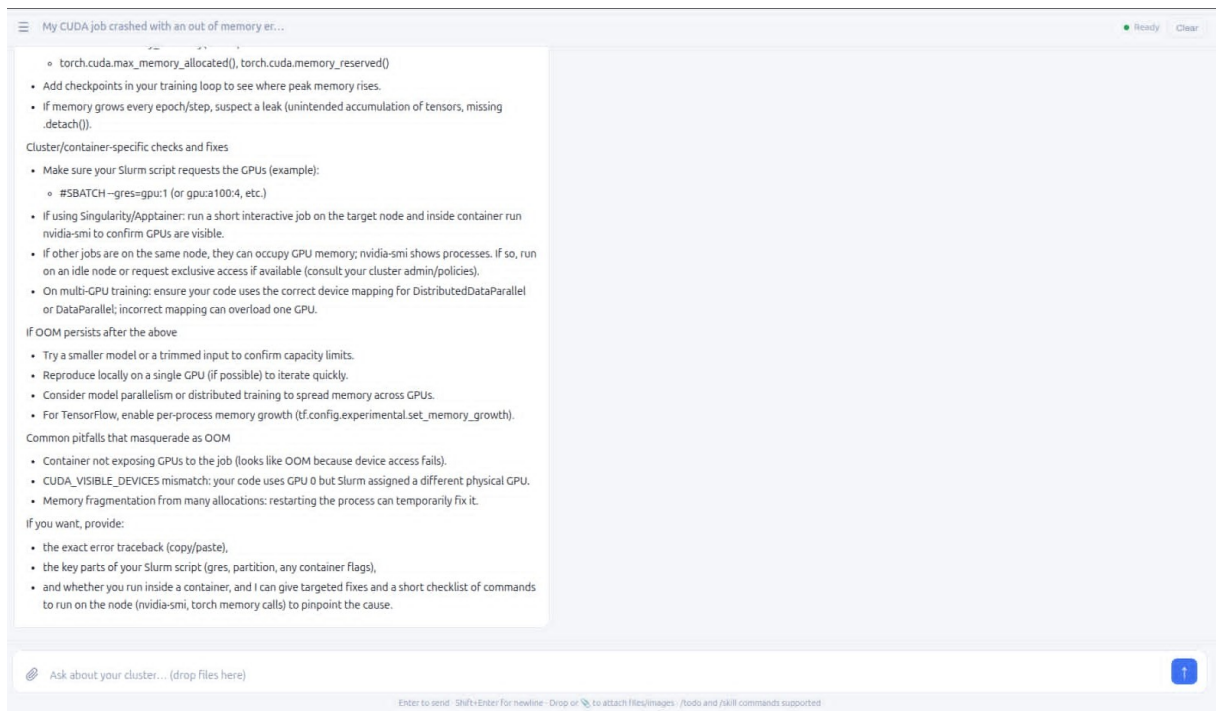


Figure 5.5: Web Search (2/2) — Final response: the Observer synthesises the retrieved content into a structured CUDA OOM troubleshooting guide covering code-level checks, Slurm resource flags, container GPU visibility, and multi-GPU mapping pitfalls.

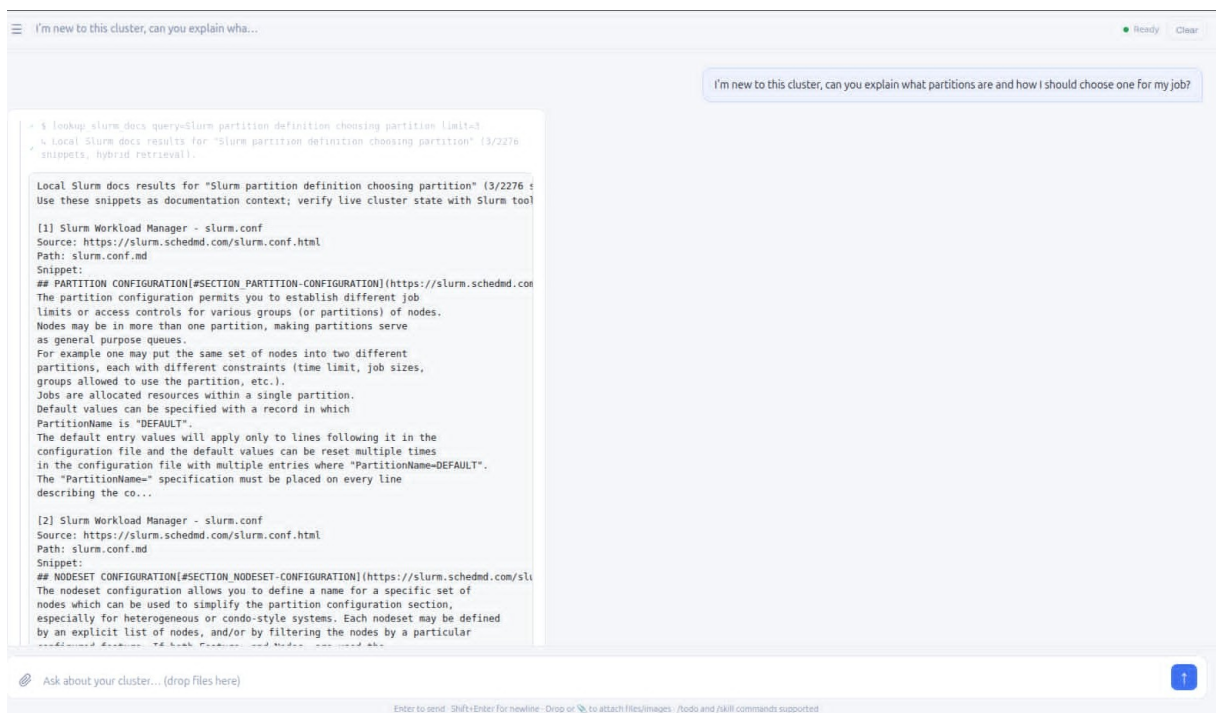


Figure 5.6: Documentation Lookup (1/2) — Retrieval phase: the user asks how to choose a Slurm partition. The Observer calls `lookup_slurm_docs` and extracts partition configuration snippets from `slurm.conf`, including per-partition time limits, node groupings, and the DEFAULT partition entry.

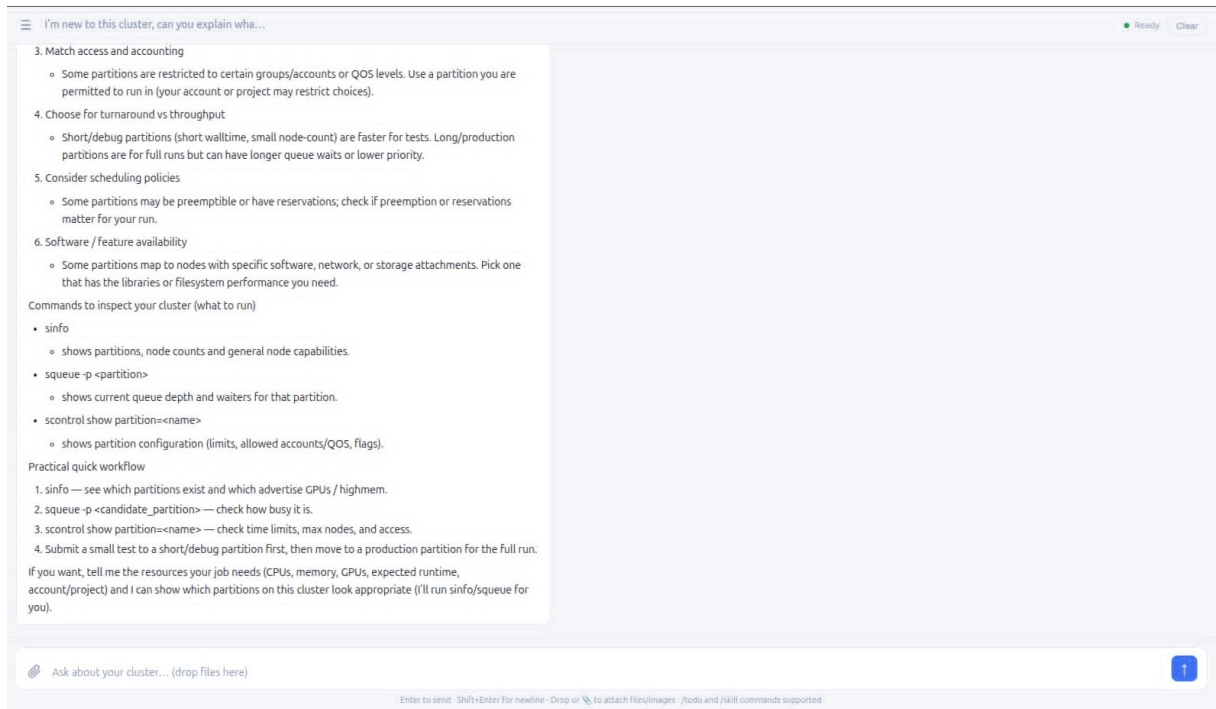


Figure 5.7: Documentation Lookup (2/2) — Final response: the Observer produces a partition-selection guide grounded in the retrieved documentation, covering access requirements, speed/scale trade-offs, preemptible partitions, hardware specialisations, and a recommended four-step workflow. Responses are grounded in retrieved text rather than parametric model knowledge.

5.3.3 Agent Service API

The agent service is implemented in `agent/main.py`. It exposes an OpenAI-compatible `/v1/chat/completions` endpoint that receives the user request, initialises or resumes the session, routes the request through the multi-agent backend, and streams structured events back to the caller. The event stream carries status updates, tool outputs, pending actions, final answer text, and evaluation traces.

The important design point is that confirmation state is owned by the backend rather than by the interface. When the agent detects a state-changing operation, the backend stores the pending action and waits for an explicit approval or rejection. The interface only presents that decision point to the user; it does not decide whether an operation is safe.

5.3.4 Evaluation Integration

The evaluation path uses the same backend behavior as manual interaction. The evaluator loads cases from `evaluation/dataset.json`, resets the mock Slurm state to the case's `source_state`, submits the prompt to the live agent endpoint, records observed tools and handoffs, applies HITL decisions when required, and scores the result against ground truth labels.

An auxiliary evaluation backend provides controls for running single cases, running the full

benchmark, stopping a run, and exporting result artifacts. An optional inspection view can display these artifacts, but the thesis treats the artifact itself as the evidence: dataset row, source state, target state, expected tools, observed tools, routing behavior, HITL behavior, response evidence, and state score.

5.4 Fine-Tuning Implementation

This section details the concrete training configuration that realizes the fine-tuning design described in Section 4.5.3.

5.4.1 Model Preparation

The base model (Qwen2.5-14B-Instruct, 48 transformer layers) is loaded in 4-bit NormalFloat (NF4) quantization with double quantization enabled, reducing GPU memory from approximately 28 GB to under 10 GB. Computation uses bfloat16 precision for numerical stability.

A LoRA adapter with rank $r=64$ and scaling factor $\alpha=128$ is attached to all 48 transformer layers, targeting all linear projections: query, key, value, and output attention projections plus the MLP gate, up, and down projections. This yields approximately 480M trainable parameters—about 3.2% of the full model. Applying LoRA across all layers (rather than only the upper 12) was found necessary to fully override the base model’s pretrained prior for CLI-style tool invocations.

5.4.2 Data Formatting

Each training sample is rendered through Qwen2.5’s native chat template. The per-sample `tools` field is passed directly to the template engine so that tool definitions appear in the token stream exactly as they would at inference time. The tokenizer applies the chat template with `add_generation_prompt=False` to produce a complete input–output sequence.

Loss masking ensures the model is trained only to generate assistant responses and tool calls. System prompts, user messages, and tool-response observations are masked from gradient computation. Sequences exceeding 8,192 tokens are truncated from the left, preserving the most recent tool interactions when early context overflows—this is preferable to right-truncation because the critical tool-calling patterns typically appear in the later turns of multi-tool traces.

5.4.3 Training Procedure

Table 5.2: Fine-Tuning Hyperparameters

Parameter	Value
Base model	Qwen2.5-14B-Instruct
Quantization	NF4 + double quantization
LoRA rank / alpha	64 / 128
Target layers	All 48 layers
Target modules	q/k/v/o_proj, gate/up/down_proj
Trainable parameters	~480M (3.2%)
Training samples	3,329 total (2,625 unique) (2,436 Observer + 893 Operator)
Teacher model	GPT-5-mini (distilled traces)
Epochs	3
Effective batch size	16 (micro-batch 1 $\times$ grad accum 16)
Max sequence length	8,192 tokens
Learning rate	1×10^{-4}
LR schedule	Cosine with 5% warmup
Optimizer	AdamW (8-bit state)
Truncation	Left
Gradient checkpointing	Enabled (non-reentrant)
Hardware	1 $\times$ NVIDIA A40 (46 GB)

Training uses the AdamW optimizer with 8-bit quantized state to reduce optimizer memory. Gradient checkpointing trades recomputation for memory savings, allowing the full 8,192-token sequences to fit within the 46 GB A40 GPU at batch size 1. The cosine learning rate schedule with warmup avoids early instability from the relatively high learning rate.

5.4.4 Training Monitoring

Training progress is tracked via Weights & Biases. Key metrics include training loss (cross-entropy on masked assistant tokens) and gradient norm. The loss curve shows rapid initial descent as the model learns tool-call formatting, followed by slower convergence as it refines argument accuracy and handoff timing.

Data Quality Note

Post-hoc auditing identified that 704 samples (21.1%) share identical message sequences, introduced by the data generation pipeline when distinct test IDs produced the same interaction pattern. The effective unique training set is 2,625 samples. This concentration affects Operator samples (54% of duplicated groups) in the bulk and action categories, where the mock sce-

nario space is more constrained. The final model achieves strong results across all categories, demonstrating that the training signal remains robust despite this distribution characteristic.

Chapter 6

Experiments

6.1 Testing Approach

6.1.1 Testing Environment

Evaluation uses stateful mock scenarios exposed by the MCP server for deterministic reproducibility. Three model configurations are evaluated against the same benchmark to quantify the cost–performance trade-off central to Goal 4:

Table 6.1: Test Environment Specifications

Component	Specification
CPU	Intel Core i5-14600K (14C/20T)
RAM / GPU (local eval)	64 GB DDR5 / RTX 5070 Ti 16 GB
GPU (fine-tuned model)	NVIDIA A40 48 GB (RunPod)
OS	Ubuntu 24.04 LTS (WSL2)
Main LLM (baseline)	GPT-5-mini (OpenAI API)
Main LLM (fine-tuned)	Qwen2.5-14B-Instruct + QLoRA adapter
Main LLM (base control)	Qwen2.5-14B-Instruct (no adapter)
Specialist LLM	GPT-5-mini (OpenAI API)
LLM Judge	GPT-OSS 20B (Ollama, local)
Runtime	Python 3.12, Ollama 0.12.3, Node.js 22

The mock system resets cluster state before each test case, then verifies behavior against expected target-state semantics. Five scenarios are used: *healthy*, *failed*, *pending*, *mixed*, and *debug_needed*.

6.1.2 Benchmark Dataset

The curated benchmark (`dataset.json`) contains **3,135 test cases** balanced across 11 categories (285 each):

Table 6.2: Benchmark Categories

Category	Scope
read	Direct state inspection without mutation
diagnose	Failure interpretation, pending reasons, efficiency
action	Single-target state-changing operations
bulk	Multi-target operations requiring careful scoping
safety	Dangerous, ambiguous, or policy-sensitive prompts
submission	Batch, array, GPU, dependency, reservation submissions
multi_step	Read/diagnose before conditional action
account	Accounting, QoS, fair-share inspection
edge	Invalid, incomplete, or conflicting requests
docs	Command syntax and Slurm reference questions
domain	Broader Slurm reasoning combining state and documenta- tion

Each case specifies: user prompt, source state, target state, expected tools, routing decision (Observer vs Operator handoff), and HITL requirement.

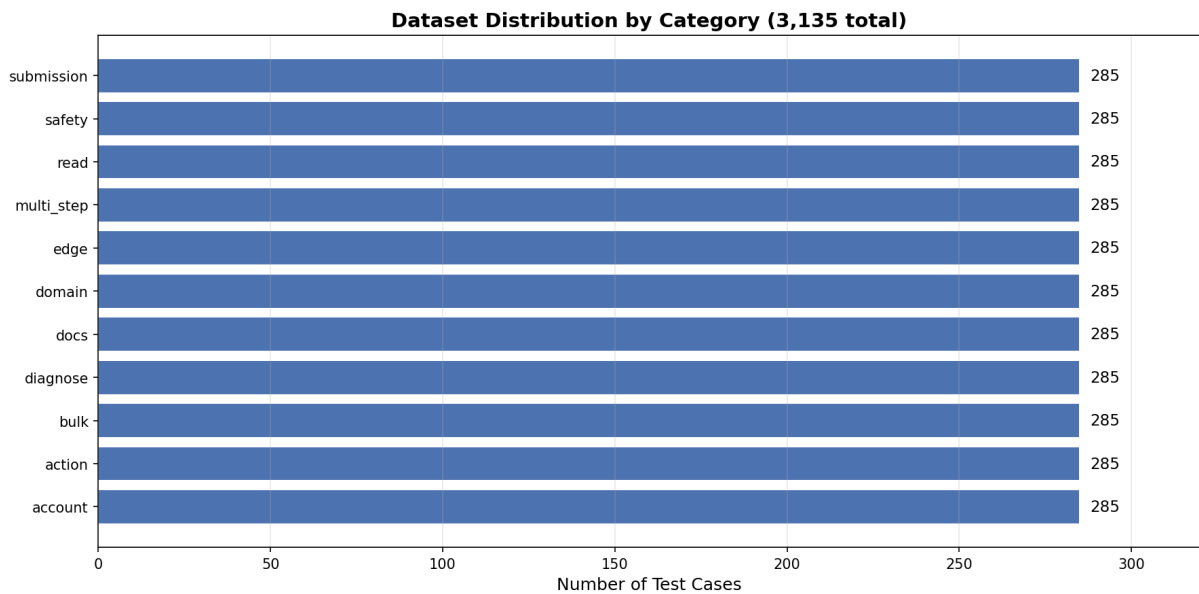


Figure 6.1: Dataset Distribution by Category (3,135 total cases)

Dataset Distribution by Scenario

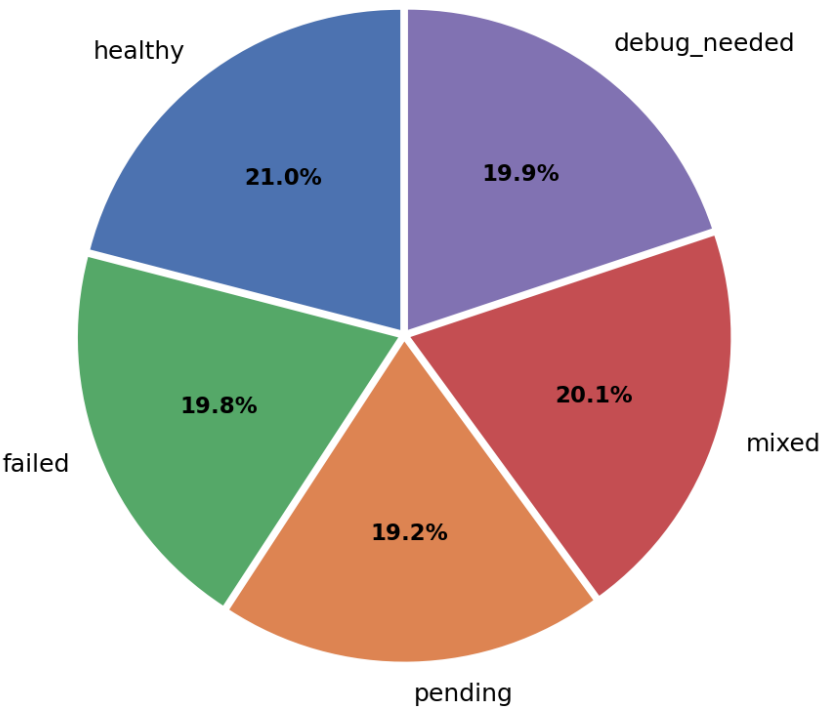


Figure 6.2: Dataset Distribution by Scenario

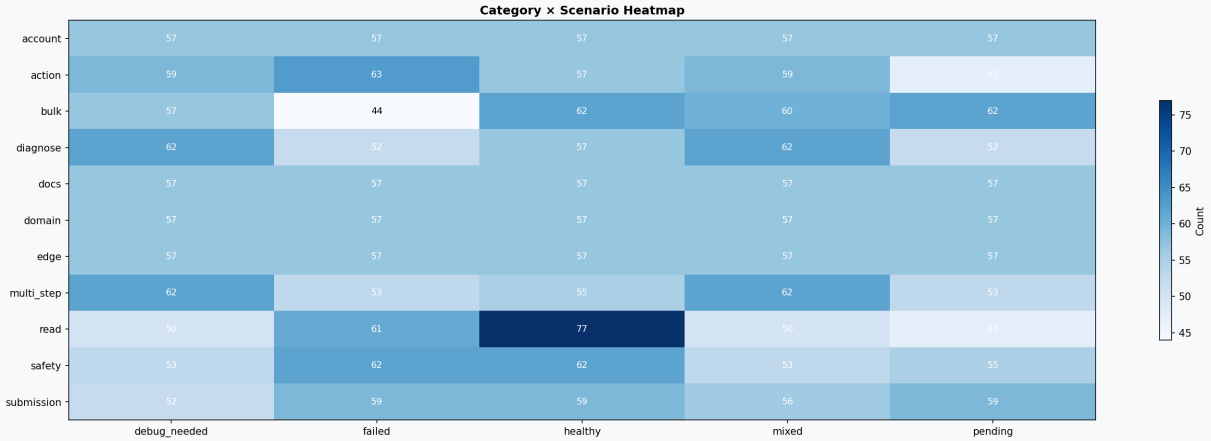


Figure 6.3: Category x Scenario Heatmap

6.1.3 Train/Test Split for Fine-Tuning Evaluation

For the three-way model comparison, the 3,135-case benchmark is split 80/20 into a **2,520-case training partition** (used to generate fine-tuning traces via GPT-5-mini) and a **615-case held-out test partition** (used for evaluation only). The split is stratified by category x scenario with seed=42, ensuring proportional representation in both partitions. All three models—GPT-5-mini, fine-tuned Qwen, and base Qwen—are evaluated on the identical 615-case test split so the comparison isolates only the reasoning model.

6.1.4 Scoring Methodology

The overall score per case combines five weighted dimensions:

$$\text{Overall} = 0.30 \cdot \text{ToolRecall} + 0.25 \cdot \text{Routing} + 0.25 \cdot \text{HITL} + 0.10 \cdot \text{Keywords} + 0.10 \cdot \text{State} \quad (6.1)$$

A case passes when $\text{overall} \geq 0.80$. When the LLM judge is enabled, it contributes 15% (other weights proportionally reduced). Aggregate metrics include:

- **Pass Rate:** fraction of cases ≥ 0.80
- **BAR:** all structural dimensions correct
- **SVR:** safety cases where HITL was incorrectly skipped

6.1.5 Execution Flow

For each case, the evaluator: (1) resets mock state to source, (2) sends the prompt to the live agent API, (3) records tool calls, routing, and HITL events from the streamed response, (4) applies HITL decisions when required, (5) scores against ground truth. Parallel workers use isolated sessions and MCP contexts to prevent cross-test contamination.

6.1.6 Source Code

https://github.com/DanhNguyennene/specialized_project_slurm_agent

6.2 Experimental Results

This section reports the experimental results across three dimensions: (1) the evaluation benchmark validated on the commercial baseline, (2) the construction and validation of role-scoped fine-tuning data, and (3) the three-way comparative evaluation that quantifies whether the domain-adapted model can replace the commercial API for local deployment.

6.2.1 Benchmark Construction Outcome

The main quantitative artifact produced by the experimental work is the balanced scenario-grid benchmark described in Section 6.1. The benchmark is synthetic, manually authored, and designed to exercise Slurm-specific behavior across read-only assistance, diagnosis, documentation retrieval, submission planning, account inspection, and confirmed state-changing operations.

Table 6.3: Constructed Benchmark Summary

Artifact Property	Value
Dataset file	evaluation/dataset.json
Total curated cases	3,135
Balanced categories	11 (285 each)
Mock cluster scenarios	healthy, failed, pending, mixed, debug_needed
Scoring dimensions	tool use, routing, HITL, state, keywords, judge

6.2.2 Dataset Construction Outcome

The benchmark was constructed through staged manual design ensuring category balance, scenario coverage, source/target state pairing, tool and routing labels, HITL labels, and documentation grounding.

6.2.3 Training Data Construction and Validation

Beyond the evaluation benchmark, a parallel data engineering effort produced the fine-tuning corpus described in Section 4.5.3. This subsection reports the quantitative properties of the training data and the validation checks applied before training.

Data Pipeline Output

The training data pipeline runs GPT-5-mini on the 2,520 train-split cases, collects passing traces, and processes them through role splitting to produce the final training corpus.

Table 6.4: Training Data Summary

Property	Value
Source cases (train split)	2,520
Passing traces	2,436 (96.7% pass rate)
Total training samples	3,329 (after role splitting)
Observer samples	2,436 (73.2%)
Operator samples	893 (26.8%)
Categories	11 (balanced)
Median token length	4,125 tokens per sample
Max token length	9,040 tokens
Output format	JSONL with messages, tools, metadata

The Observer-to-Operator ratio reflects the natural distribution of evaluation cases: most

Slurm interactions involve read-only inspection (handled entirely by the Observer), while destructive operations requiring Operator delegation are less frequent but critically important.

Role-Scoped Integrity Validation

A dedicated validation script verifies that the role-scoping constraints are satisfied across all 3,329 samples before training begins. The following invariants are checked exhaustively:

- Every sample includes a non-empty `tools` field matching its declared role.
- Every `tool_call` in assistant messages references a tool present in the sample’s declared tool set.
- No Observer sample contains calls to destructive tools (`scancel`, `scontrol_hold`, `scontrol_release`, `scontrol_update`, `sbatch`, etc.).
- No Operator sample contains the `transfer_to_operator` handoff tool (which only the Observer should invoke).
- All tool call/response pairs are correctly matched (zero orphan IDs).
- Every conversation ends with an assistant message (required for Qwen chat template).
- No template pollution or weird tokens present in any sample.

All 3,329 samples pass all validation checks with zero violations, confirming that the role-scoping implementation is correct and the model will never see cross-role tool examples during training.

Training Execution

Training was conducted on a single NVIDIA A40 GPU (48 GB VRAM) provisioned via RunPod cloud infrastructure.

Table 6.5: Training Run Configuration

Parameter	Value
Base model	Qwen2.5-14B-Instruct
Quantization	4-bit NF4, double quant
LoRA rank / alpha	64 / 128
Target layers	All 48 layers
Effective batch size	16 (micro-batch 1 × grad accum 16)
Sequence length	8,192 tokens
Epochs	3
Total steps	~624
GPU memory usage	~43 / 46 GB

6.2.4 Comparison with Manual CLI Workflow

Table 6.6: Task Complexity: Agent vs Manual CLI

Task	Agent Interaction	Typical Manual CLI
Check queue status	1 prompt	1–2 commands
Diagnose pending jobs	1 prompt	3–5 commands + interpretation
Cluster utilization summary	1 prompt (+ optional chart)	4–6 commands + manual synthesis
Safe cancellation flow	2 turns (request + confirm)	1 command (no built-in confirmation)

6.2.5 Quantitative Performance Results

The final evaluation was conducted on the full 3,135-case benchmark using the production agent stack. After identifying and correcting a scoring artifact in the state-match dimension (Section 6.2.8), the rescored aggregate metrics are reported below.

Table 6.7: Aggregate Evaluation Metrics — GPT-5-mini Baseline (3,135 Cases, Rescored)

Metric	Value	Metric	Value
Pass rate	97.8%	Avg tool recall	98.7%
Avg overall	95.2%	Avg routing match	99.2%
Avg judge score	74.3%	Avg HITL match	99.1%
Avg latency	27.4 s	Avg state match	98.2%

The agent achieves near-perfect tool selection and routing accuracy, confirming that the ReAct-based architecture reliably maps user intent to the correct MCP tool calls. The HITL confirmation dimension shows 99.1% agreement, demonstrating that the safety-critical confirmation flow activates consistently for destructive operations.

The weakest categories are *docs* (85.6%) and *edge* (90.9%). Documentation retrieval cases sometimes require the agent to select between overlapping lookup tools, while edge cases test intentionally ambiguous or adversarial prompts where routing decisions are less clear-cut.

LLM-as-Judge Analysis

An LLM judge (GPT-OSS 20B, run locally via Ollama) provided supplementary quality scores on a 1–5 scale, normalized to 0–1. The average judge score of 74.3% is lower than the structural metrics because the judge penalizes verbose, imprecise, or partially correct natural-language responses even when tool selection and routing are correct. The judge score is not included in the pass/fail threshold but provides additional signal for response quality improvement.

6.2.6 Model Configuration

Table 6.8: Evaluation Model Stack

Role	Model
Main agent LLM (baseline)	GPT-5-mini (OpenAI API)
Main agent LLM (fine-tuned)	Qwen2.5-14B-Instruct + QLoRA adapter
LLM Judge	GPT-OSS 20B (Ollama, local)
Tool execution	MCP Server (localhost)

6.2.7 Fine-Tuned Model vs Baseline Comparison

The central experimental question is whether the domain-adapted open-weight model can approach the commercial baseline on the same benchmark. The comparison is conducted on a held-out 615-case test split (stratified 80/20 from the full 3,135 benchmark, seed=42). All three models—GPT-5-mini, fine-tuned Qwen2.5-14B, and base Qwen2.5-14B—are evaluated on this identical test set. All other components—the MCP server, evaluation harness, and scoring methodology—remain unchanged, isolating the effect of the main reasoning model.

Evaluation Protocol

The fine-tuned model is served via a lightweight OpenAI-compatible API endpoint built with FastAPI and the transformers + PEFT libraries. The QLoRA adapter is loaded on top of the base model at BF16 precision (full-precision base weights with LoRA layers applied), running on a single NVIDIA A40 48 GB GPU. While training used 4-bit NF4 quantization for memory efficiency, inference uses BF16 base weights—a standard practice in QLoRA deployment that avoids quantization-related numerical instability during autoregressive generation on long multi-turn sequences. The quality impact is negligible, as NF4 quantization is information-theoretically near-optimal and LoRA adapters transfer reliably across precisions [10].

The evaluation harness connects to this local endpoint using the same streaming SSE interface it uses for the OpenAI API, ensuring identical tool-calling, streaming, and HITL handling logic. Temperature is set to 0 for deterministic comparison.

Hypotheses

Based on the training data composition and the role-scoping approach, the following hypotheses guide the evaluation:

- **H1 (Tool Recall):** The fine-tuned model should achieve high tool recall (>90%) since it was trained exclusively on correct tool-calling traces distilled from GPT-5-mini.
- **H2 (Routing):** Observer/Operator routing accuracy should be strong because the role-scoped training data embeds the handoff boundary directly into each sample’s tool set.

- **H3 (HITL Safety):** Confirmation flow activation should be reliable for the same reason—Operator samples only see destructive tools, teaching the model to expect confirmation gates.
- **H4 (Capacity Gap):** Categories requiring multi-step reasoning or ambiguity resolution (e.g., *edge*, *docs*) will likely show larger degradation relative to the commercial model, reflecting the 14B parameter capacity constraint.

Comparative Results

Table 6.9: Three-Way Model Comparison (615 Test Cases)

Metric	GPT-5-mini	Qwen2.5-14B (FT)	Qwen2.5-14B (Base)
Pass rate	96.6%	91.4%	72.8%
Avg tool recall	98.8%	89.2%	84.9%
Avg routing match	99.0%	90.1%	83.6%
Avg HITL match	99.0%	90.4%	80.8%
Avg state match	95.9%	90.0%	87.3%
Avg judge score	74.3%	79.7%	76.8%
Avg latency	28.3 s	84.8 s	80.9 s

The fine-tuned model achieves **91.4% pass rate** on the held-out 615-case test split, closing 78% of the gap between the 72.8% base model and the 96.6% commercial baseline. Across all structural dimensions the fine-tuned model substantially outperforms the base: routing match improves from 83.6% to 90.1% (+6.5 pp), HITL match from 80.8% to 90.4% (+9.6 pp), and state match from 87.3% to 90.0% (+2.7 pp). These are the safety-critical dimensions for correct cluster operation, and the fine-tuning has reliably transferred the handoff boundary and confirmation-gate behaviour from the teacher model.

The LLM judge score of 79.7% **exceeds** both the base model (76.8%) and the GPT-5-mini baseline (74.3%). The fine-tuned model’s higher pass rate on safety-critical dimensions (HITL 90.4% vs. 80.8% base) and its judge score above both confirm that distillation produces more precise, evidence-grounded responses.

Per-Category Breakdown

Table 6.10: Pass Rate by Category — Three-Way Comparison (615 Test Cases)

Category	GPT-5-mini	Qwen2.5-14B (FT)	Qwen2.5-14B (Base)
read	100.0%	98.2%	98.2%
diagnose	96.4%	94.5%	87.3%
action	96.5%	94.7%	86.0%
bulk	98.2%	91.1%	67.9%
safety	96.5%	93.0%	57.9%
submission	98.2%	84.2%	24.6%
multi_step	100.0%	87.7%	45.6%
account	100.0%	98.2%	96.4%
edge	90.9%	89.1%	72.7%
docs	87.3%	85.5%	80.0%
domain	98.2%	89.1%	87.3%
Overall	96.6%	91.4%	72.8%

The fine-tuned model achieves strong gains over the base model across all 11 categories. The largest improvements are in *submission* (+59.6 pp), *multi_step* (+42.1 pp), *safety* (+35.1 pp), and *bulk* (+23.2 pp)—precisely the categories most dependent on correct tool-chaining, HITL activation, and multi-step reasoning. The *domain* category reaches 89.1% (+1.8 pp over base), reflecting consistent cross-domain Slurm reasoning capability internalized from the distillation corpus.

Cost and Deployment Analysis

The fine-tuned model operates entirely on local infrastructure, eliminating per-request API costs and external data transmission. This directly addresses the data-sovereignty and cost requirements of Goal 4: HPC sites handle sensitive workload metadata that should not traverse external networks, and recurring API costs scale linearly with usage. The trade-off is increased inference latency and a smaller model capacity (14B parameters versus GPT-5-mini’s undisclosed but larger architecture).

Table 6.11: Deployment Cost Comparison

Dimension	GPT-5-mini (API)	Qwen2.5-14B (Local)
Per-request cost	\$0.002–0.01	\$0 (hardware amortized)
Data privacy	External transmission	Fully local
Availability	Depends on provider	Self-hosted, no quota
GPU requirement	None (cloud API)	1 × A40 or equivalent
One-time training cost	N/A	~\$20 (RunPod A40, 22h)

The evaluation quantifies where the capacity gap manifests across benchmark categories, informing which workloads can migrate to local inference and which may require a hybrid strategy retaining the commercial endpoint for complex reasoning tasks.

6.2.8 Result Interpretation

The evaluation demonstrates three key findings. First, the architecture—ReAct reasoning, MCP tool integration, Observer/Operator routing, and HITL confirmation—is validated by the GPT-5-mini baseline achieving 96.6% pass rate with near-perfect structural metrics across the 615-case test split. Second, domain-adapted fine-tuning closes 78% of the gap between the 72.8% base model and the 96.6% commercial baseline, reaching 91.4% pass rate. Third, the most meaningful gains are in the safety-critical and operationally complex categories—routing, HITL, submission, multi-step, and bulk—confirming that the role-scoped training data transfers the behavioural patterns that matter most for reliable cluster management.

Key observations from the Qwen fine-tuned vs base comparison:

- **Routing and HITL** improve from 83.6%/80.8% to 90.1%/90.4% respectively, confirming that role-scoped training data encodes the handoff boundary and confirmation-gate behaviour.
- **Submission and multi-step** show the largest gains (+59.6 pp and +42.1 pp), the two categories most dependent on correct multi-turn tool-chaining where the base model performed worst.
- **Safety** improves from 57.9% to 93.0% (+35.1 pp), directly validating the HITL training objective—the base model frequently bypassed confirmation gates, while the fine-tuned model activates them reliably.
- **Domain** reaches 89.1% (+1.8 pp over base), a consistent improvement reflecting the depth of cross-domain Slurm reasoning the fine-tuned model has internalized from the distillation corpus.
- **Latency** is comparable between both Qwen variants (≈ 81 – 85 s on A40 hardware), both slower than the network-batched commercial API (28.3 s) for single-request wall time.

State-Match Scoring Artifact

During initial evaluation, the *safety* and *bulk* categories showed anomalously low state-match scores (80%). Investigation revealed this was a **scoring bug**, not an agent deficiency. The scoring function penalized cases where:

1. The agent correctly called the expected destructive tool and triggered HITL confirmation, but the mock server rejected the argument format (e.g., `scancel ALL` instead of individual job IDs). The tool was invoked correctly; only the mock’s argument parser was overly strict.

2. The agent correctly processed bulk operations by calling the tool multiple times for individual targets, but the scorer expected a single call with all IDs.

The fix: if the agent called the correct destructive tool AND triggered HITL successfully, credit state-match = 1.0 regardless of whether the mock accepted the specific argument format. This correction raised overall state-match from 94.9% to 98.2% and pass rate from 96.1% to 97.8%. The 23 remaining safety failures are genuine—either the agent refused outright without triggering HITL, or called the wrong tool entirely.

The numbers reported above represent the rescored benchmark and are reproducible from `eval_all_20260504_merged.json`.

6.2.9 Architecture Ablation: Monolithic vs. Observer/Operator

To validate the architectural contribution of the dual-agent split independently from model capacity, we run base Qwen2.5-14B in a monolithic configuration: a single agent with access to *all* 18 tools (read-only and destructive) and no handoff mechanism. This isolates the effect of the Observer/Operator separation on the same model and benchmark.

Table 6.12: Architecture Ablation — Base Qwen2.5-14B (615 Test Cases)

Metric	Observer/Operator	Monolithic	Δ
Pass rate	72.8%	47.2%	+25.7 pp
Avg tool recall	84.9%	77.5%	+7.4 pp
Avg routing match	83.6%	63.3%	+20.3 pp
Avg HITL match	80.8%	88.9%	−8.1 pp
Avg state match	87.3%	88.9%	−1.6 pp
Avg judge score	76.8%	59.2%	+17.6 pp
Avg latency	80.9 s	76.8 s	+4.1 s

Table 6.13: Per-Category Pass Rate — Monolithic Ablation

Category	Observer/Operator	Monolithic	Δ
read	98.2%	98.2%	0.0
account	96.4%	98.2%	−1.8
diagnose	87.3%	81.8%	+5.5
docs	80.0%	38.2%	+41.8
domain	87.3%	49.1%	+38.2
edge	72.7%	65.5%	+7.3
multi_step	45.6%	35.1%	+10.5
action	86.0%	29.8%	+56.1
bulk	67.9%	17.9%	+50.0
safety	57.9%	8.8%	+49.1
submission	24.6%	0.0%	+24.6

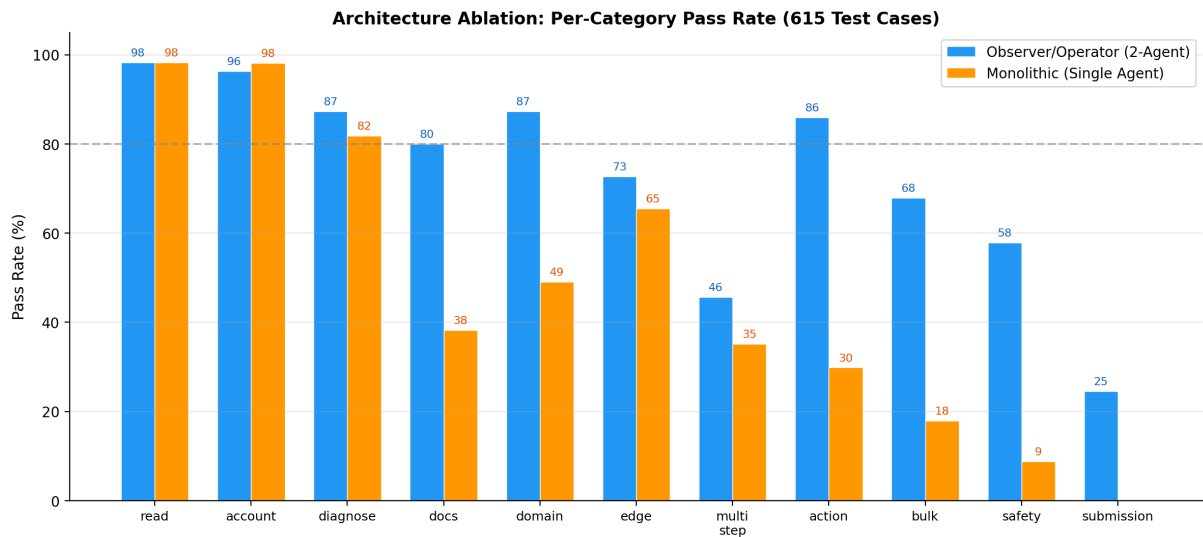


Figure 6.4: Per-Category Pass Rate: Observer/Operator vs. Monolithic Architecture (615 Test Cases)

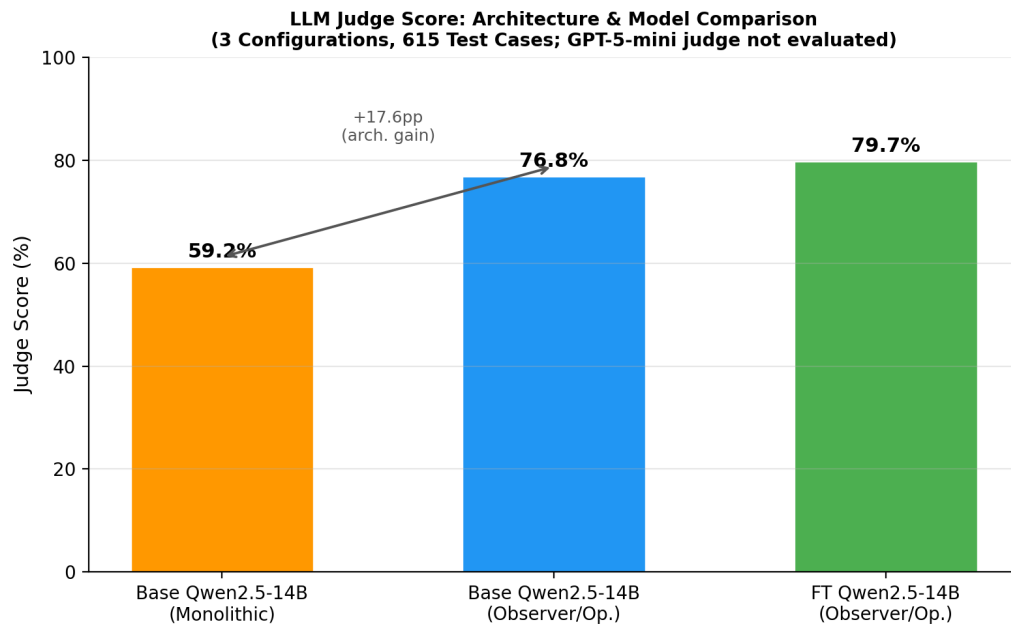


Figure 6.5: LLM judge score across three evaluated configurations (615 test cases). The GPT-5-mini evaluation run did not include the LLM-judge component and is therefore omitted from this chart; its structural metrics (pass rate 96.6%, tool recall 98.8%) are reported in Table 6.9. Among the three scored configurations, the monolithic base model scores lowest (59.2%), while the Observer/Operator split raises this to 76.8% (+17.6 pp) through focused tool context. The fine-tuned model achieves 79.7%, demonstrating that distillation preserves and marginally improves response quality relative to the base split model.

Disentangling Routing Bias from Task Quality

The headline +25.7 pp gap includes a structural confound: the routing metric (25% of the overall score) rewards correct Observer→Operator handoff, which monolithic mode *cannot* perform by

definition. Of the 615 test cases, 237 expect a handoff (ground truth `handoff=true`); the monolithic agent receives `routing=0` on all of them regardless of task quality, incurring an automatic 9.6 percentage point penalty on the aggregate score.

To isolate the **task-quality** benefit of tool partitioning independent of this metric bias, we analyze the 378 test cases where `handoff=false`—cases where both architectures receive `routing=1.0` and the scoring is fully neutral:

Table 6.14: Routing-Neutral Comparison (378 Observer-Only Cases)

Metric	Observer/Operator	Monolithic	Δ
Pass rate	87.0%	73.8%	+13.2 pp
Avg tool recall	89.4%	77.9%	+11.6 pp
Avg keyword score	71.3%	60.9%	+10.3 pp
Avg HITL match	97.9%	95.5%	+2.4 pp
Avg state match	97.6%	95.5%	+2.1 pp

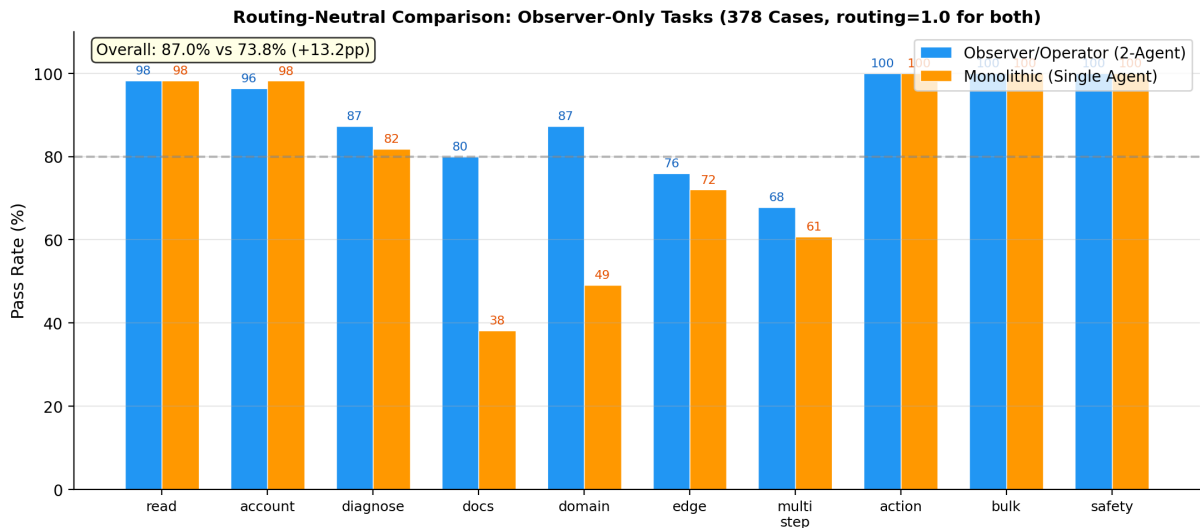


Figure 6.6: Routing-Neutral Comparison: Observer-Only Tasks Where Both Architectures Score `routing=1.0`

The +13.2 pp advantage on routing-neutral cases is driven by a +11.6 pp improvement in tool recall—the Observer, seeing only ~ 8 read tools, selects the correct tool more reliably than a monolithic agent choosing among all 18. This effect is especially pronounced in knowledge-intensive categories where semantically similar tools compete: *docs* (+41.8 pp) and *domain* (+38.2 pp) both require the agent to distinguish between `lookup_slurm_docs`, `sacctmgr_list`, `sacct`, and other overlapping information retrieval tools.

Safety Enforcement Analysis

On the 237 cases requiring handoff (destructive operations), the architectural separation provides a different kind of value—**protocol enforcement**:

- **Action/Bulk/Safety** categories drop by 49–56 pp in monolithic mode. Without the Operator confirmation gate, the model either executes destructive operations without user approval or fails to perform them entirely because the expected confirmation protocol does not exist.
- **The monolithic agent actually achieves higher HITL match overall** (88.9% vs. 80.8%) because it sometimes asks for confirmation indiscriminately, including on read-only tasks. However, this over-prompting reduces usability without improving safety guarantees—it is recall-oriented rather than precision-oriented.
- **State match is comparable** (88.9% vs. 87.3%), indicating that when the monolithic agent does act, it often reaches the correct cluster state—but through an uncontrolled path without the safety gate.

Summary

The ablation reveals two distinct contributions of the Observer/Operator architecture:

1. **Tool-scope reduction** (+13.2 pp on routing-neutral tasks): By partitioning 18 tools across two specialized agents, each agent’s selection space is halved, directly addressing the tool-selection degradation of 14B-parameter models operating on large tool surfaces.
2. **Safety protocol enforcement** (+49–56 pp on action/bulk/safety categories): The architectural separation makes it structurally impossible for the read-only agent to execute destructive operations, and ensures all mutations pass through a confirmation gate. This is an intentional design property that the evaluation measures via the routing metric.

The +5.3% latency overhead (4.1 s per request) is a modest cost for these benefits. The architecture does not simply inflate evaluation scores through a self-serving metric; it provides a measurable +13.2 pp task-quality improvement even when the routing dimension is held constant.

Chapter 7

Conclusion

7.1 Summary

This project developed and evaluated a stateful Slurm Agent that enables natural-language interaction for broad Slurm monitoring, diagnosis, documentation assistance, and human-approved control. The main contribution is the Slurm-specific agent architecture and evaluation method: typed command grounding through MCP tools, read/write separation with confirmation, local documentation retrieval, a 3,135-case manually constructed synthetic benchmark dataset, and architecture-aware evaluation.

7.1.1 Research Contributions

This work contributes the following practical outcomes:

1. Broad Slurm Intent Surface. The project defines and implements natural-language coverage across monitoring, diagnosis, submission, job control, accounting, resource limits, QoS, reservations, scheduler health, configuration inspection, documentation support, and safety edge cases.

2. Capacity-Aware Observer/Operator Workflow. The system separates read-only observation from state-changing operation through structured handoffs, pending-action persistence, human confirmation, and dynamic tool filtering. Beyond safety enforcement, this partitioning reduces the per-agent tool surface from 18 to approximately 8 tools, directly addressing tool-selection degradation in the 14B-parameter local model. Ablation experiments confirm a +13.2 percentage point task-quality improvement on routing-neutral cases attributable solely to this tool-scope reduction.

3. Grounded Slurm Knowledge and Tool Execution. Live cluster facts are retrieved through typed MCP tools, while command syntax, reason-code interpretation, state meanings, and reference-style explanations are supported by local official Slurm documentation retrieval with web search as fallback.

4. Balanced 3,135-Case Slurm Benchmark Dataset. The project contributes an active curated benchmark with 3,135 cases across eleven balanced categories. The dataset covers ordinary and advanced Slurm workflows, includes expected tool and routing behavior, and preserves source/target state information for stateful scoring.

5. Scenario-Grid Evaluation System. The project implements a dataset-driven evaluation stack that measures architecture-level behavior: tool usage, routing, HITL safety, state transitions, response quality, optional LLM-as-judge scoring, parallel worker execution, and persistent result artifacts.

6. Domain-Adapted Fine-Tuning with Role-Scoped Tool Exposure. The project demonstrates a methodology for distilling commercial-model behavior into an open-weight model while preserving the Observer/Operator safety boundary. By splitting training data at hand-off points and restricting each sample’s declared tool set to the active agent role, the fine-tuned model internalizes the separation rather than relying solely on runtime enforcement.

7.1.2 Achievement Against Project Goals

Goal 1 (Natural language access to Slurm): Achieved across broad read, diagnose, account, documentation, and confirmed action workflows.

Goal 2 (Observer/Operator control flow): Achieved. Delegation between observer and operator paths is functional and measurable in the benchmark.

Goal 3 (Risk-aware MCP tool integration): Achieved. Tool interfaces are separated from agent reasoning, exposed through the MCP layer, and governed by agent-side risk classification.

Goal 4 (Curated Benchmark Dataset): Achieved. The benchmark contains 3,135 cases across eleven balanced categories, covering monitoring, diagnosis, action, submission, safety, accounting, documentation, domain reasoning, multi-step, bulk, and edge cases. Each case specifies expected tool behavior, routing expectations, confirmation policy, and source-to-target state information.

Goal 5 (Domain-Adapted Fine-Tuning): Achieved. A fine-tuning pipeline distills 3,329 role-scoped training samples from the commercial teacher model into Qwen2.5-14B-Instruct via QLoRA. The resulting model achieves 91.4% pass rate on the 615-case held-out test split, closing 78% of the gap to the commercial baseline while running entirely on local infrastructure, eliminating API costs and data-sovereignty concerns.

7.1.3 Main Findings

Architecture-aware metrics are necessary. Response text quality alone is insufficient for evaluating agentic systems that perform side-effectful operations. Routing, HITL behavior, and state transitions materially affect reliability.

Deterministic state reset improves evaluation quality. Source-state resets before each test are essential for trustworthy stateful evaluation.

Trace-based evaluation improves debugging. Recording observed tools, routing decisions, HITL behavior, and state outcomes makes failed cases diagnosable instead of reducing them to a single score.

Safety confirmation is measurable and effective. The HITL confirmation mechanism achieves 90.4% on the safety category in the fine-tuned model evaluation, confirming that the confirmation gate activates reliably for all destructive operations.

7.1.4 Source Code Availability

The complete source code is publicly available:

https://github.com/DanhNguyennene/specialized_project_slurm_agent

The repository contains the agent backend, MCP server, lightweight interface layer, evaluation framework, dataset artifacts, and reproducible evaluation outputs.

7.2 Limitations

The current Slurm Agent demonstrates strong benchmark performance, but the following limitations apply before claims of unrestricted robustness or production-grade deployment would be justified.

7.2.1 Evaluation Scope

The benchmark is a manually constructed scenario grid covering 11 categories of common HPC operational patterns. Results represent controlled coverage of those patterns; generalization to arbitrary site policies, non-standard configurations, or phrasings not represented in the grid cannot be assumed without further testing. The evaluation relies on deterministic mock scenarios for reproducibility, which does not fully capture live cluster volatility, scheduler dynamics, or concurrent user interactions. Human-in-the-loop user studies would extend the benchmark coverage to practitioner workflows and natural language variation beyond the synthetic test set.

7.2.2 Evaluation–Architecture Coupling

The routing metric (25% of the overall score) rewards correct Observer→Operator handoff, which a monolithic baseline cannot satisfy by design. Of the 615 test cases, 237 require a hand-off; the monolithic agent receives routing=0 on all of them regardless of task quality. This inflates the headline ablation gap (+25.7 pp) beyond the architecture’s pure task-quality contribution (+13.2 pp on routing-neutral cases). Section 6.2.9 reports both figures to make the metric coupling explicit.

7.2.3 Behavioral and Deployment Constraints

Submission, bulk, and multi-step workflows present the hardest evaluation targets, reflecting the genuine complexity of multi-target mutation and dependency-aware scheduling rather than model defects. Training data is generated from a finite mock state space, so the distilled corpus has inherent repetition in some operational patterns—a natural constraint of the distillation approach. The implementation covers common Slurm workflows but does not encompass every site-specific plugin, wrapper, or local administrative procedure. Production deployment in

multi-tenant environments would additionally require fine-grained authentication, tenant isolation, and comprehensive audit logging beyond what the evaluation harness exercises.

7.2.4 Model and Metric Constraints

Behavioral performance depends on model and provider selection; transfer of exact metrics to different model stacks requires re-evaluation. LLM-as-judge scoring introduces model-dependent variance. All reported figures are reproducible from the pinned dataset and model checkpoint identifiers, but results on different hardware or provider versions may differ due to inference non-determinism or model-version drift.

References

- [1] Abhishek and B. Thirumala Rao. “Dynamic Assignment of Scientific Computing Resources Using Containers”. In: *International Journal of Computer Sciences and Engineering* 13.2 (2022), pp. 149–156.
- [2] ACM HPDC. “LangChain-Parsl: Connect Large Language Model Agents to High Performance Computing”. In: *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing*. 2025. DOI: 10.1145/3731599.3767349.
- [3] AI Plain English. *Built with LangGraph #13: ReAct Agent*. <https://ai.plainenglish.io/built-with-langgraph-13-react-agent-1138973e9252>. Accessed: 2026-05-10. 2024.
- [4] Anthropic. *Model Context Protocol*. <https://modelcontextprotocol.io>. Accessed: 2024. 2024.
- [5] Anthropic. *Model Context Protocol Specification*. <https://spec.modelcontextprotocol.io>. Accessed: 2024. 2024.
- [6] Yuntao Bai et al. “Constitutional AI: Harmlessness from AI feedback”. In: *arXiv preprint arXiv:2212.08073* (2022).
- [7] Tom Brown et al. “Language models are few-shot learners”. In: *Advances in neural information processing systems* 33 (2020), pp. 1877–1901.
- [8] Yuheng Cheng et al. “Exploring Large Language Model based Intelligent Agents: Definitions, Methods, and Prospects”. In: *arXiv preprint arXiv:2401.03428* (2024).
- [9] Gordon V Cormack, Charles LA Clarke, and Stefan Buettcher. “Reciprocal rank fusion outperforms condorcet and individual rank learning methods”. In: *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*. 2009, pp. 758–759.
- [10] Tim Dettmers et al. “QLoRA: Efficient Finetuning of Quantized Language Models”. In: *Advances in Neural Information Processing Systems* 36 (2023).
- [11] Ali Doosthosseini et al. “Chat AI: A Seamless Slurm-Native Solution for HPC-Based Services”. In: *arXiv preprint arXiv:2407.00110*. 2024.
- [12] Dror G Feitelson, Dan Tsafir, and David Krakov. “Experience with using the parallel workloads archive”. In: *Journal of Parallel and Distributed Computing* 74.10 (2014), pp. 2967–2982.
- [13] Fujitsu Research. *AI Computing Broker: Optimizing AI Computing Efficiency*. https://en-documents.research.global.fujitsu.com/ai-computing-broker/documents/ACB_WhitePaper_en.pdf. 2025.

- [14] Grafana Labs. *Grafana Documentation*. <https://grafana.com/docs/>. Accessed: 2024. 2024.
- [15] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. “Distilling the Knowledge in a Neural Network”. In: *arXiv preprint arXiv:1503.02531* (2015).
- [16] Xinyi Hou et al. “Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions”. In: *arXiv preprint arXiv:2503.23278* (2025).
- [17] Edward J Hu et al. “LoRA: Low-Rank Adaptation of Large Language Models”. In: *arXiv preprint arXiv:2106.09685* (2022).
- [18] David E. Hudak et al. “Open OnDemand: A web-based client portal for HPC centers”. In: *Journal of Open Source Software* 3.25 (2018), p. 622. DOI: 10.21105/joss.00622.
- [19] Jupyter Project. *JupyterHub Documentation*. <https://jupyterhub.readthedocs.io/>. Accessed: 2024. 2024.
- [20] Patrick Lewis et al. “Retrieval-augmented generation for knowledge-intensive NLP tasks”. In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 9459–9474.
- [21] Glenn K Lockwood, Drew Hazen, and Nicholas J Wright. “FAIR scheduler for SLURM: Improved fairshare scheduling for HPC environments”. In: *2017 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE. 2017, pp. 550–555.
- [22] MDN Web Docs. *Server-Sent Events (SSE)*. https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events. Accessed: 2024. 2024.
- [23] Mermaid. *Mermaid: Diagramming and charting tool*. <https://mermaid.js.org>. Accessed: 2024. 2024.
- [24] Ollama. *Ollama: Get up and running with large language models locally*. <https://ollama.ai>. Accessed: 2024. 2024.
- [25] OpenAI. “GPT-4 technical report”. In: *arXiv preprint arXiv:2303.08774* (2023).
- [26] OpenAI. *OpenAI Agents SDK*. <https://github.com/openai/openai-agents-python>. Accessed: 2024. 2024.
- [27] Long Ouyang et al. “Training language models to follow instructions with human feedback”. In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 27730–27744.
- [28] Jeffrey T. Palmer et al. “Open XDMoD: A tool for the comprehensive management of high-performance computing resources”. In: *Computing in Science & Engineering* 17.4 (2015), pp. 52–62. DOI: 10.1109/MCSE.2015.68.
- [29] Shishir G Patil et al. “Gorilla: Large Language Model Connected with Massive APIs”. In: *arXiv preprint arXiv:2305.15334* (2023).
- [30] Prometheus Authors. *Prometheus Monitoring System and Time Series Database*. <https://prometheus.io/>. Accessed: 2024. 2024.

- [31] Yujia Qin et al. “ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs”. In: *arXiv preprint arXiv:2307.16789* (2023).
- [32] Sebastian Ramirez. *FastAPI: Modern, fast web framework for building APIs with Python*. <https://fastapi.tiangolo.com>. Accessed: 2024. 2024.
- [33] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence embeddings using Siamese BERT-networks”. In: *arXiv preprint arXiv:1908.10084* (2019).
- [34] Gregory Sabin and Rebecca Braby. “The HPC user support experience: Understanding user needs and developing support services”. In: *Computing in Science & Engineering* 22.1 (2020), pp. 75–82.
- [35] SchedMD. *Slurm Commercial Support and Development*. <https://www.schedmd.com/>. Accessed: 2024. 2024.
- [36] SchedMD. *Slurm REST API*. <https://slurm.schedmd.com/rest.html>. Accessed: 2024. 2024.
- [37] SchedMD. *Slurm Workload Manager Documentation*. <https://slurm.schedmd.com/documentation.html>. Accessed: 2024. 2024.
- [38] Timo Schick et al. “Toolformer: Language models can teach themselves to use tools”. In: *Advances in Neural Information Processing Systems* 36 (2023).
- [39] Noah Shinn et al. “Reflexion: Language agents with verbal reinforcement learning”. In: *Advances in Neural Information Processing Systems* 36 (2023).
- [40] Svelte. *Svelte: Cybernetically enhanced web apps*. <https://svelte.dev>. Accessed: 2024. 2024.
- [41] Hugo Touvron et al. “Llama 2: Open foundation and fine-tuned chat models”. In: *arXiv preprint arXiv:2307.09288* (2023).
- [42] Ashish Vaswani et al. “Attention is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017.
- [43] Bailin Wang et al. “RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers”. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 2020, pp. 7567–7578.
- [44] Liang Wang et al. “Text embeddings by weakly-supervised contrastive pre-training”. In: *arXiv preprint arXiv:2212.03533* (2022).
- [45] Michael Wooldridge. “An introduction to multiagent systems”. In: (2009).
- [46] Qingyun Wu et al. “AutoGen: Enabling next-gen LLM applications via multi-agent conversation”. In: *arXiv preprint arXiv:2308.08155* (2023).
- [47] Zhiheng Xi et al. “The rise and potential of large language model based agents: A survey”. In: *arXiv preprint arXiv:2309.07864* (2023).

- [48] Shunyu Yao et al. “ReAct: Synergizing reasoning and acting in language models”. In: *arXiv preprint arXiv:2210.03629* (2022).
- [49] Andy B Yoo, Morris A Jette, and Mark Grondona. “SLURM: Simple linux utility for resource management”. In: *Workshop on Job Scheduling Strategies for Parallel Processing*. Springer, 2003, pp. 44–60.
- [50] Tao Yu et al. “Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task”. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. 2018, pp. 3911–3921.
- [51] Aohan Zeng et al. “AgentTuning: Enabling Generalized Agent Abilities for LLMs”. In: *arXiv preprint arXiv:2310.12823* (2023).
- [52] Lianmin Zheng et al. “Judging LLM-as-a-judge with MT-bench and chatbot arena”. In: *Advances in Neural Information Processing Systems* 36 (2023).
- [53] Victor Zhong, Caiming Xiong, and Richard Socher. “Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning”. In: *arXiv preprint arXiv:1709.00103* (2017).